
AN EXPERT’S GUIDE TO TRAINING PHYSICS-INFORMED NEURAL NETWORKS

Sifan Wang

Graduate Group in Applied Mathematics
and Computational Science
University of Pennsylvania
Philadelphia, PA 19104
sifanw@sas.upenn.edu

Shyam Sankaran

Department of Mechanical Engineering
and Applied Mechanics
University of Pennsylvania
Philadelphia, PA 19104
shyamss@seas.upenn.edu

Hanwen Wang

Graduate Group in Applied Mathematics
and Computational Science
University of Pennsylvania
Philadelphia, PA 19104
wangh19@sas.upenn.edu

Paris Perdikaris

Department of Mechanical Engineering
and Applied Mechanics
University of Pennsylvania
Philadelphia, PA 19104
pgp@seas.upenn.edu

ABSTRACT

Physics-informed neural networks (PINNs) have been popularized as a deep learning framework that can seamlessly synthesize observational data and partial differential equation (PDE) constraints. Their practical effectiveness however can be hampered by training pathologies, but also oftentimes by poor choices made by users who lack deep learning expertise. In this paper we present a series of best practices that can significantly improve the training efficiency and overall accuracy of PINNs. We also put forth a series of challenging benchmark problems that highlight some of the most prominent difficulties in training PINNs, and present comprehensive and fully reproducible ablation studies that demonstrate how different architecture choices and training strategies affect the test accuracy of the resulting models. We show that the methods and guiding principles put forth in this study lead to state-of-the-art results and provide strong baselines that future studies should use for comparison purposes. To this end, we also release a highly optimized library in JAX that can be used to reproduce all results reported in this paper, enable future research studies, as well as facilitate easy adaptation to new use-case scenarios.

1 Introduction

Recent advances in deep learning have revolutionized fields such as computer vision, natural language processing and reinforcement learning [1, 2, 3]. Powered by rapid growth in computational resources, deep neural networks are also increasingly used for modeling and simulating physical systems. Examples of these include weather forecasting [4, 5, 6], quantum chemistry [7, 8] and protein structure prediction [9].

Notably, the fusion of scientific computing and machine learning has led to the emergence of physics-informed neural networks (PINNs) [10], an emerging paradigm for tackling forward and inverse problems involving partial differential equations (PDEs). These deep learning models are known for their capability to seamlessly incorporate noisy experimental data and physical laws into the learning process. This is typically accomplished by parameterizing unknown functions of interest using deep neural networks and formulating a multi-task learning problem with the aim of matching observational data and approximating an underlying PDE system. Over the last couple of years, PINNs have led to a series of promising results across a range of problems in computational science and engineering, including fluids mechanics [11, 12, 13], bio-engineering [14, 15], materials [16, 17, 18], molecular dynamics [19], electromagnetics [20, 21], geosciences [22, 23], and the design of thermal systems [24, 25].

Despite some empirical success, PINNs are still facing many challenges that define open areas for research and further methodological advancements. In recent years, there have been numerous studies focusing on improving the performance of PINNs, mostly by designing more effective neural network architectures or better training algorithms. For example, loss re-weighting schemes have emerged as a prominent strategy for promoting a more balanced training process and improved test accuracy [26, 27, 28, 29]. Other efforts aim to achieve similar goals by adaptively re-sampling collocation points, such as importance sampling [30], evolutionary sampling [31] and residual-based adaptive sampling [32]. Considerable efforts have also been dedicated towards developing new neural network architectures to improve the representation capacity of PINNs. Examples include the use of adaptive activation functions [33], positional embeddings [34, 35], and novel architectures [26, 36, 37, 38, 39, 40]. Another research avenue explores alternative objective functions for PINNs training, beyond the weighted summation of residuals [41]. Some approaches incorporate numerical differentiation [42], while others draw inspiration from Finite Element Methods (FEM), adopting variational formulations [43, 44]. Other approaches propose adding additional regularization terms to accelerate training of PINNs [45, 46]. Lastly, the evolution of training strategies has been an area of active research. Techniques such as sequential training [47, 48] and transfer learning [49, 50, 51] have shown potential in speeding up the learning process and yielding better predictive accuracy.

While new research on PINNs is currently being produced at high frequency, a suite of common benchmarks and baselines is still missing from the literature. Almost all existing studies put forth their own collection of benchmark examples, and typically compare against the original PINNs formulation put forth by Raissi *et al.*, which is admittedly a weak baseline. This introduces many difficulties in systematically assessing progress in the field, but also in determining how to use PINNs from a practitioner’s standpoint.

To address this gap, this work proposes a training pipeline that seamlessly integrates recent research developments to effectively resolve the identified issues in PINNs training, including spectral bias [52, 35], unbalanced back-propagated gradients [26, 27] and causality violation [53]. In addition, we present a variety of techniques that could further enhance performance, shedding light on some practical tips that form a guideline for selecting hyper-parameters. This is accompanied by an extensive suite of fully reproducible ablation studies performed across a wide range of benchmarks. This allows us to identify the setups that consistently yield the state-of-the-art results, which we believe should become the new baseline that future studies should compare against. We also release a high-performance library in JAX that can be used to reproduce all findings reported in this work, enable future research studies, as well as facilitate easy adaptation to new use-case scenarios. As such, we believe that this work can equally benefit researchers and practitioners to further advance PINNs and deploy them in more realistic application settings.

The rest of this paper is organized as follows. In section 2, we provide a brief overview of the original formulation of PINNs as introduced by Raissi *et al.* [10], and outline our training pipeline. From Section 3 to Section 5, we delve into the motivation and implementation details of the key components of the proposed algorithm. These consist of non-dimensionalization, network architectures that employ Fourier feature embeddings and random weight factorization, as well as training algorithms such as causal training, curriculum training and loss weighting strategies. Section 6 discusses various aspects of PINNs that lead to improved stability and superior training performance. Finally, in section 7 we validate the effectiveness and robustness of the proposed pipeline across a wide range of benchmarks and showcase state-of-the-art results.

2 Physics-informed Neural Networks

Following the original formulation of Raissi *et al.*, we begin with a brief overview of physics-informed neural networks (PINNs) [10] in the context of solving partial differential equations (PDEs). Generally, we consider PDEs taking the form

$$\mathbf{u}_t + \mathcal{N}[\mathbf{u}] = 0, \quad t \in [0, T], \quad \mathbf{x} \in \Omega, \quad (2.1)$$

subject to the initial and boundary conditions

$$\mathbf{u}(0, \mathbf{x}) = \mathbf{g}(\mathbf{x}), \quad \mathbf{x} \in \Omega, \quad (2.2)$$

$$\mathcal{B}[\mathbf{u}] = 0, \quad t \in [0, T], \quad \mathbf{x} \in \partial\Omega, \quad (2.3)$$

where $\mathcal{N}[\cdot]$ is a linear or nonlinear differential operator, and $\mathcal{B}[\cdot]$ is a boundary operator corresponding to Dirichlet, Neumann, Robin, or periodic boundary conditions. In addition, $\mathbf{u}$ describes the unknown latent solution that is governed by the PDE system of Equation (2.1).

We proceed by representing the unknown solution $\mathbf{u}(t, \mathbf{x})$ by a deep neural network $\mathbf{u}_\theta(t, \mathbf{x})$, where θ denotes all tunable parameters of the network (e.g., weights and biases). This allows us to define the PDE residuals as

$$\mathcal{R}_\theta(t, \mathbf{x}) = \frac{\partial \mathbf{u}_\theta}{\partial t}(t, \mathbf{x}) + \mathcal{N}[\mathbf{u}_\theta](t, \mathbf{x}) \quad (2.4)$$

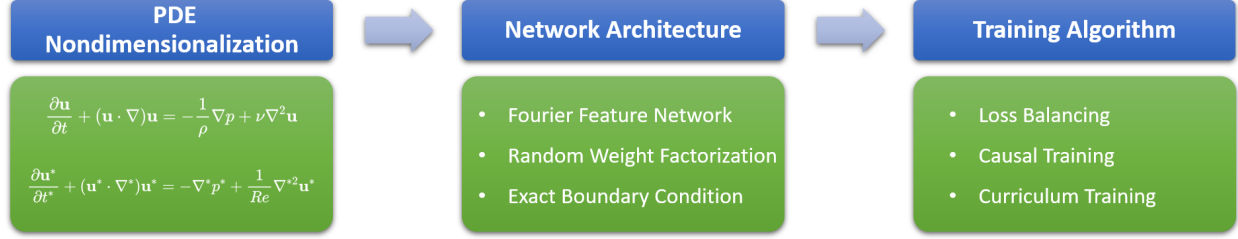


Figure 1: Illustration of the proposed training pipeline. The procedure begins with the non-dimensionalization of the PDE system, ensuring that input and output variables are in a reasonable range. Subsequently, an appropriate network architecture is constructed to represent the unknown PDE solution. The use of Fourier feature embeddings and random weight factorization is highly recommended for mitigating spectral bias and accelerating convergence. The training phase of the PINN model integrates various advanced algorithms, including self-adaptive loss balancing, causal training, and curriculum training.

Then, a physics-informed model can be trained by minimizing the following composite loss function

$$\mathcal{L}(\theta) = \mathcal{L}_{ic}(\theta) + \mathcal{L}_{bc}(\theta) + \mathcal{L}_r(\theta), \quad (2.5)$$

where

$$\mathcal{L}_{ic}(\theta) = \frac{1}{N_{ic}} \sum_{i=1}^{N_{ic}} |\mathbf{u}_\theta(0, \mathbf{x}_{ic}^i) - \mathbf{g}(\mathbf{x}_{ic}^i)|^2, \quad (2.6)$$

$$\mathcal{L}_{bc}(\theta) = \frac{1}{N_{bc}} \sum_{i=1}^{N_{bc}} |\mathcal{B}[\mathbf{u}_\theta](t_{bc}^i, \mathbf{x}_{bc}^i)|^2, \quad (2.7)$$

$$\mathcal{L}_r(\theta) = \frac{1}{N_r} \sum_{i=1}^{N_r} |\mathcal{R}_\theta(t_r^i, \mathbf{x}_r^i)|^2. \quad (2.8)$$

Here $\{\mathbf{x}_{ic}^i\}_{i=1}^{N_{ic}}$, $\{t_{bc}^i, \mathbf{x}_{bc}^i\}_{i=1}^{N_{bc}}$ and $\{t_r^i, \mathbf{x}_r^i\}_{i=1}^{N_r}$ can be the vertices of a fixed mesh or points that are randomly sampled at each iteration of a gradient descent algorithm. Notice that all required gradients with respect to input variables or network parameters θ can be efficiently computed via automatic differentiation [54].

However, as demonstrated by recent work, several critical training pathologies prevent PINNs from yielding accurate and robust results. These pathologies include spectral bias [52, 35], causality violation [53], and unbalanced back-propagated gradients among different loss terms [26], etc. To address these issues, we propose a training pipeline that integrates key recent advancements, which we believe are indispensable for the successful implementation of PINNs. As shown in Figure 1, the pipeline consists of three main steps, PDE non-dimensionalization, choosing suitable network architectures and employing appropriate training algorithms. Further details are provided in Algorithm 2. In the following sections, we will carefully demonstrate the motivation and necessity of each component in the proposed algorithm and validate its effectiveness via a wide range of benchmarks.

3 Non-dimensionalization

It is well-known that data normalization is an important pre-processing step in traditional deep learning, which typically involves scaling the input features of a data-set so that they have similar magnitudes and ranges [55, 56]. However, this process may not be generally applicable for PINNs as the target solutions are typically not available when solving forward PDE problems. In such cases, it is important to ensure that the target output variables vary within a reasonable range. One way to achieve this is through non-dimensionalization. It is a common technique used in mathematics and physics, to simplify and analyze complex systems by transforming the original system into an equivalent dimensionless system. This is performed by selecting one or more fundamental units or characteristic values, and scaling the variables in the problem so that they become dimensionless and of order one. From our experience, non-dimensionalization plays a crucial role in building physics-informed models especially for dealing with experimental data or real-world problems. The reasons are shown below:

- Lack of consistent network initialization schemes: The initialization of neural networks has a crucial role on the effectiveness of gradient descent algorithms. Common initialization schemes (e.g., Glorot [55]) not

Algorithm 1 Training pipeline of physics-informed neural networks

1. Non-dimensionalize the PDE system (2.1).
2. Represent the PDE solution by a multi-layer perceptron network (MLP) $\mathbf{u}_\theta$ with Fourier feature embeddings and random weight factorization. In general, we recommend using tanh activation and initialized using the Glorot scheme.
3. Formulate the weighted loss function according to the PDE system:

$$\mathcal{L}(\theta) = \lambda_{ic}\mathcal{L}_{ic}(\theta) + \lambda_{bc}\mathcal{L}_{bc}(\theta) + \lambda_r\mathcal{L}_r(\theta), \quad (2.9)$$

where $\mathcal{L}_{ic}(\theta)$ and $\mathcal{L}_{bc}(\theta)$ are defined in (2.6), (2.7) respectively, and

$$\mathcal{L}_r(\theta) = \frac{1}{M} \sum_{i=1}^M w_i \mathcal{L}_r^i(\theta). \quad (2.10)$$

Here we partition the temporal domain into M equal sequential segments and introduce $\mathcal{L}_r^i$ to denote the PDE residual loss within the i -th segment of the temporal domain.

4. Set all global weights $\lambda_{ic}, \lambda_{bc}, \lambda_r$ and temporal weights $\{w_i\}_{i=1}^M$ to 1.

5. Use S steps of a gradient descent algorithm to update the parameters θ as:

for $n = 1, \dots, S$ **do**

(a) Randomly sample $\{\mathbf{x}_{ic}^i\}_{i=1}^{N_{ic}}, \{t_{bc}^i, \mathbf{x}_{bc}^i\}_{i=1}^{N_{bc}}$ and $\{t_r^i, \mathbf{x}_r^i\}_{i=1}^{N_r}$ in the computational domain and evaluated each loss terms $\mathcal{L}_{ic}, \mathcal{L}_{bc}$ and $\{\mathcal{L}_r^i\}_{i=1}^M$.

(b) Compute and update the temporal weights by

$$w_i = \exp\left(-\epsilon \sum_{k=1}^{i-1} \mathcal{L}_r^k(\theta)\right), \text{ for } i = 2, 3, \dots, M. \quad (2.11)$$

Here $\epsilon > 0$ is a user-defined hyper-parameter that determines the "slope" of temporal weights.

if $n \bmod f = 0$ **then**

(c) Compute the global weights by

$$\hat{\lambda}_{ic} = \frac{\|\nabla_\theta \mathcal{L}_{ic}(\theta)\| + \|\nabla_\theta \mathcal{L}_{bc}(\theta)\| + \|\nabla_\theta \mathcal{L}_r(\theta)\|}{\|\nabla_\theta \mathcal{L}_{ic}(\theta)\|}, \quad (2.12)$$

$$\hat{\lambda}_{bc} = \frac{\|\nabla_\theta \mathcal{L}_{ic}(\theta)\| + \|\nabla_\theta \mathcal{L}_{bc}(\theta)\| + \|\nabla_\theta \mathcal{L}_r(\theta)\|}{\|\nabla_\theta \mathcal{L}_{bc}(\theta)\|}, \quad (2.13)$$

$$\hat{\lambda}_r = \frac{\|\nabla_\theta \mathcal{L}_{ic}(\theta)\| + \|\nabla_\theta \mathcal{L}_{bc}(\theta)\| + \|\nabla_\theta \mathcal{L}_r(\theta)\|}{\|\nabla_\theta \mathcal{L}_r(\theta)\|}, \quad (2.14)$$

where $\|\cdot\|$ denotes the L^2 norm.

(d) Update the global weights $\lambda = (\lambda_{ic}, \lambda_{bc}, \lambda_r)$ using a moving average of the form

$$\lambda_{\text{new}} = \alpha \lambda_{\text{old}} + (1 - \alpha) \hat{\lambda}_{\text{new}}. \quad (2.15)$$

where the parameter α determines the balance between the old and new values

end if

(e) Update the parameters θ via gradient descent

$$\theta_{n+1} = \theta_n - \eta \nabla_\theta \mathcal{L}(\theta_n) \quad (2.16)$$

$$(2.17)$$

end for

The recommended default values for hyper-parameters are as follows: $f = 1,000, \alpha = 0.9, \gamma = 1.0, \epsilon = 1.0$. Please note that we freeze the back-propagation of the weights w_i 's and λ_i 's with respect to network parameters θ .

only prevent vanishing gradients but also accelerate training convergence. A critical assumption for these initialization methods is that input variables should be in a moderate range, such as having a zero mean and unit variance, which enables smooth and stable forward and backward propagation. To satisfy this assumption, we propose using non-dimensionalization to scale the input and output variables so that they are of order one.

- Mitigating the disparities in variable scales: If input and output variables have different scales, some can dominate over others, leading to unbalanced contributions in the model training, therefore hindering the learning of meaningful correlations between them. Non-dimensionalization, which scales variables to have similar magnitudes and ranges, can help to reduce this discrepancy and facilitate model training.
- Improving convergence: If the variables are not properly scaled, the optimization algorithm may have to take very small steps to adjust the weights for one variable while large steps for another variable. This may result in a slow and unstable training process. Through non-dimensionalization, the optimizer can take more consistent steps, yielding faster convergence and better performance.

While non-dimensionalization is an indispensable pre-processing step, it is not a “silver bullet” that can resolve all issues in training PINNs. One of the main differences between PINNs and conventional deep learning tasks is the minimization of PDE residuals, which introduces additional difficulties in optimization process. Even if all variables are properly scaled via non-dimensionalization, the scale of PDE residuals can still vastly differ from the scale of the latent solution function, leading to a considerable discrepancy in the scale of different loss terms. Therefore, it is important to carefully inspect and re-scale the loss terms that define the PINNs objective. In section 5.2, we introduce two self-adaptive loss weighting schemes based on the magnitude of back-propagated gradients and Neural Tangent Kernel (NTK) theory. We will show that these methods can automatically balance the interplay between different loss terms during training and lead to more robust model performance.

4 Network Architecture

In this section, we delve into the selection of suitable network architectures for training PINNs. We begin by providing a brief overview of multi-layer perceptrons, along with common hyper-parameter choices, activation functions, and initialization schemes. Then, we discuss random Fourier feature embeddings, a simple yet effective technique that enables coordinate MLPs to learn complex high frequency functions. Finally, we introduce random weight factorization, a simple drop-in replacement of dense layers that has been shown to consistently accelerate training convergence and improve model performance.

4.1 Multi-layer Perceptrons (MLP)

We mainly use multi-layer perceptrons (MLPs) as a universal approximator to represent the latent functions of interest, which takes the coordinates of a spatio-temporal domain as inputs and predicts the corresponding target solution functions. Specifically, let $\mathbf{x} \in \mathbb{R}^d$ be the input, $\mathbf{g}^{(0)}(\mathbf{x}) = \mathbf{x}$ and $d_0 = d$. A MLP $f_\theta(\mathbf{x})$ is recursively defined by

$$\mathbf{f}^{(l)}(\mathbf{x}) = \mathbf{W}^{(l)} \cdot \mathbf{g}^{(l-1)}(\mathbf{x}) + \mathbf{b}^{(l)}, \quad \mathbf{g}^{(l)}(\mathbf{x}) = \sigma(\mathbf{f}^{(l)}(\mathbf{x})), \quad l = 1, 2, \dots, L, \quad (4.1)$$

with a final layer

$$\mathbf{f}_\theta(\mathbf{x}) = \mathbf{W}^{(L+1)} \cdot \mathbf{g}^{(L)}(\mathbf{x}) + \mathbf{b}^{(L+1)}, \quad (4.2)$$

where $\mathbf{W}^{(l)} \in \mathbb{R}^{d_l \times d_{l-1}}$ is the weight matrix in l -th layer and σ is an element-wise activation function. Here, $\theta = (\mathbf{W}^{(1)}, \mathbf{b}^{(1)}, \dots, \mathbf{W}^{(L+1)}, \mathbf{b}^{(L+1)})$ represents all trainable parameters in the network.

In practice, the choice of an appropriate network architecture impacts the success of physics-informed neural networks. From our experience, networks that are too narrow and shallow lack the expressive capacity to capture complex nonlinear functions, while networks that are too wide and deep can be difficult to optimize. Therefore, we recommend employing networks with width and depth ranging from 128 to 512 and 3 to 6, respectively, which tends to yield relatively optimal and robust results. To build a continuously differentiable neural representation, we recommend using the hyperbolic tangent (Tanh). Other popular choices include sinusoidal functions [36] and GeLU [57]. We point out that ReLU is not suitable since its second-order derivative is zero, which inevitably saturates the computation of PDE residuals. Finally, dense layers will be typically initialized using the Glorot scheme [55].

4.2 Random Fourier features

As demonstrated by [52, 58, 59], MLPs suffer from a phenomenon referred to as spectral bias, showing that they are biased towards learning low frequency functions. This undesired preference also prevents PINNs from learning

high frequencies and fine structures of target solutions [35]. In Appendix A, we present a detailed analysis of this phenomenon via a lens of Neural Tangent Kernel (NTK) theory,

To mitigate spectral bias, Tancik *et al.* [60] proposed random Fourier feature embeddings, which map input coordinates into high frequency signals before passing through a MLP. This encoding $\gamma : \mathbb{R}^n \rightarrow \mathbb{R}^m$ is defined by

$$\gamma(\mathbf{x}) = \begin{bmatrix} \cos(\mathbf{B}\mathbf{x}) \\ \sin(\mathbf{B}\mathbf{x}) \end{bmatrix}, \quad (4.3)$$

where each entry in $\mathbf{B} \in \mathbb{R}^{m \times d}$ is sampled from a Gaussian distribution $\mathcal{N}(0, \sigma^2)$ and $\sigma > 0$ is a user-specified hyper-parameter.

This simple method has been shown to significantly enhance the performance of PINNs in approximating sharp gradients and complex solutions [35]. It is worth emphasizing the significance of the scale factor σ in the performance of neural networks. As demonstrated in Appendix A and [35], this hyper-parameter directly governs the frequencies of γ_i and the resulting eigenspace of the NTK, thereby biasing the network to learn certain band-limited signals. Specifically, lower encoding frequencies can result in blurry predictions, while higher encoding frequencies can introduce salt-and-pepper artifacts. Ideally, an appropriate σ should be selected such that the band width of NTK matches that of the target signals. This not only accelerates the training convergence, but also improves the prediction accuracy. However, the spectral information of the solution may not be accessible when solving forward PDEs. In practice, we recommend using a moderately large $\sigma \in [1, 10]$.

4.3 Random weight factorization

Recently, Wang *et al.* [61] proposed random weight factorization (RWF) and demonstrated that it can consistently improve the performance of PINNs. RWF factorizes the weights associated with each neuron in the network as

$$\mathbf{w}^{(k,l)} = \mathbf{s}^{(k,l)} \cdot \mathbf{v}^{(k,l)}, \quad (4.4)$$

for $k = 1, 2, \dots, d_l$, $l = 1, 2, \dots, L + 1$, where $\mathbf{w}^{(k,l)} \in \mathbb{R}^{d_{l-1}}$ is a weight vector representing the k -th row of the weight matrix $\mathbf{W}^{(l)}$, $\mathbf{s}^{(k,l)} \in \mathbb{R}$ is a trainable scale factor assigned to each individual neuron, and $\mathbf{v}^{(k,l)} \in \mathbb{R}^{d_{l-1}}$. Consequently, the proposed weight factorization can be written by

$$\mathbf{W}^{(l)} = \text{diag}(\mathbf{s}^{(l)}) \cdot \mathbf{V}^{(l)}, \quad l = 1, 2, \dots, L + 1. \quad (4.5)$$

with $\mathbf{s}^{(l)} \in \mathbb{R}^{d_l}$.

We provide a geometric intuition of weight factorization in Appendix B. More theoretical and experimental results can be found in Appendix B and [61].

In practice, RWF is applied as follows. We first initialize the parameters of an MLP using a standard scheme, e.g. Glorot scheme [55]. Then, for every weight matrix $\mathbf{W}$, we proceed by initializing a scale vector $\exp(\mathbf{s})$ where $\mathbf{s}$ is sampled from a multivariate normal distribution $\mathcal{N}(\mu, \sigma \mathbf{I})$. Then every weight matrix is factorized by the associated scale factor as $\mathbf{W} = \text{diag}(\exp(\mathbf{s})) \cdot \mathbf{V}$ at initialization. Finally, we apply gradient descent to the new parameters $\mathbf{s}$, $\mathbf{V}$ directly. This procedure is summarized in Algorithm 2. The use of exponential parameterization is motivated by Weight Normalization [62] to strictly avoid zeros or very small values in the scale factors and allow them to span a wide range of different magnitudes. Empirically, too small μ, σ values may lead to performance that is similar to a conventional MLP, while too large μ, σ can result in an unstable training process. Therefore, we recommend setting $\mu = 0.5$ or 1 , and $\sigma = 0.1$, which seem to consistently and robustly improve the loss convergence and model accuracy.

Algorithm 2 Random weight factorization (RWF)

1. Initialize a neural network f_θ with $\theta = \{\mathbf{W}^{(l)}, \mathbf{b}^{(l)}\}_{l=1}^{L+1}$ (e.g. using the Glorot scheme [55]).
 - for** $l = 1, 2, \dots, L$ **do**
 - (a) Initialize each scale factor as $\mathbf{s}^{(l)} \sim \mathcal{N}(\mu, \sigma \mathbf{I})$.
 - (b) Construct the factorized weight matrices as $\mathbf{W}^{(l)} = \text{diag}(\exp(\mathbf{s}^{(l)})) \cdot \mathbf{V}^{(l)}$.
 - end for**
 2. Train the network by gradient descent on the factorized parameters $\{\mathbf{s}^{(l)}, \mathbf{V}^{(l)}, \mathbf{b}^{(l)}\}_{l=1}^{L+1}$.
The recommended hyper-parameters are $\mu = 1.0, \sigma = 0.1$.
-

5 Training

5.1 Respecting Temporal Causality

In this section, we discuss the motivation and details of equation (2.11) in Algorithm 1. Recently, Wang *et al.* [53] illustrates that PINNs may violate temporal causality when solving time-dependent PDEs, and hence are susceptible to converge towards erroneous solutions. This is mainly because the conventional PINNs tend to minimize all PDE residuals simultaneously meanwhile they are undesirably biased toward minimizing PDE residuals at later time, even before obtaining the correct solutions for earlier times. A more detailed analysis can be found in Appendix C and [53].

To impose the missing causal structure within the optimization process, we first split the temporal domain into M equal sequential segments and introduce $\mathcal{L}_r^i$ to denote the PDE residual loss within the i -th segment of the temporal domain. Then the original PDE residual loss can be rewritten as

$$\mathcal{L}_r(\theta) = \frac{1}{M} \sum_{i=1}^M w_i \mathcal{L}_r^i(\theta). \quad (5.1)$$

Combing with equation (2.11), we obtain

$$\mathcal{L}_r(\theta) = \frac{1}{M} \sum_{i=1}^M \exp\left(-\epsilon \sum_{k=1}^{i-1} \mathcal{L}_r^k(\theta)\right) \mathcal{L}_r^i(\theta). \quad (5.2)$$

It can be observed that w_i is inversely exponentially proportional to the magnitude of the cumulative residual loss from the previous time steps. As a result, $\mathcal{L}_r^i(\theta)$ will not be minimized unless all previous residuals $\{\mathcal{L}_r^k(\theta)\}_{k=1}^{i-1}$ decrease to sufficiently small value such that w_i is large enough. These temporal weights encourage PINNs to the PDE solution progressively along the time axis, in accordance with how the information propagates in time, as the dynamics evolve throughout the spatio-temporal domain.

We emphasize that the computational cost of calculating temporal weights is negligible, as the temporal weights are computed by directly evaluating the PINNs loss functions, whose values are already stored in the computational graph during training. Moreover, it is important to note that the temporal weights are functions of the trainable parameters θ . In our JAX implementation [63], we use `lax.stop_gradient` to avoid the computation of back-propagated gradients of temporal weights, thereby further conserving computational resources.

We must point out that the proposed weighted residual loss does exhibit some sensitivity to the causality parameter ϵ . Choosing a very small ϵ may fail to impose enough temporal causality, resulting in the PINN model behaving similarly to the conventional one. Conversely, choosing a large ϵ value can result in a more difficult optimization problem, because the temporal residuals at earlier times have to decrease to a very small value in order to activate the latter temporal weights. Achieving this may be difficult in some cases due to limited network capacity for minimizing the target residuals. In practice, we recommend choosing a moderately large ϵ to ensure that all temporal weights can properly converge to 1 at the end of training. If this is not the case, it would be advisable to slightly reduce the value of ϵ .

5.2 Loss Balancing

As mentioned in Section 3, one of the main challenges in training PINNs is addressing multi-scale losses that arise from the minimization of PDE residuals. These losses cannot be normalized during the pre-processing step. An alternative approach involves assigning appropriate weights to each loss term to scale them during training. However, manually choosing weights is impractical, as the optimal weights can vary greatly across different problems, making it difficult to find a fixed empirical recipe that is transferable to various PDEs. More importantly, since the solution to a PDE is unknown, there is no validation data-set available for fine-tuning these hyper-parameters in the context of solving PDEs.

Given that, our training pipeline integrates a self-adaptive learning rate annealing algorithm, which can automatically balance losses during training. Specifically, we first compute $\hat{\lambda}$ by equation (2.12)-(2.14). Then we obtain

$$\|\hat{\lambda}_{ic} \nabla_{\theta} \mathcal{L}_{ic}(\theta)\| = \|\hat{\lambda}_{bc} \nabla_{\theta} \mathcal{L}_{bc}(\theta)\| = \|\hat{\lambda}_r \nabla_{\theta} \mathcal{L}_{ic}(\theta)\| = \|\nabla_{\theta} \mathcal{L}_{ic}(\theta)\| + \|\nabla_{\theta} \mathcal{L}_{bc}(\theta)\| + \|\nabla_{\theta} \mathcal{L}_r(\theta)\| \quad (5.3)$$

This effectively guarantees that the norm of gradients of each weighted loss is equal to each other, preventing our model from being biased towards minimizing certain terms during training. The actual weights are then updated as a running average of their previous values, as defined by Equation (2.15). This technique mitigates the instability of stochastic gradient descent. In practice, these updates can either take place every hundred or thousand iterations of the gradient

descent loop or at a user-specified frequency. Consequently, the extra computational overhead associated with these updates is negligible, particularly when updates are infrequent.

We now introduce an alternative weighting scheme that leverages the resulting NTK matrix of PINNs [27]. To this end, we define the NTK matrices corresponding to $\mathcal{L}_{ic}$, $\mathcal{L}_{bc}$, and $\mathcal{L}_r$ as follows:

$$K_{ic} = \left[\left\langle \frac{\partial u}{\partial \theta}(0, \mathbf{x}_{ic}^i), \frac{\partial u}{\partial \theta}(0, \mathbf{x}_{ic}^j) \right\rangle \right], \quad (5.4)$$

$$K_{bc} = \left[\left\langle \frac{\partial u}{\partial \theta}(t_{bc}^i, \mathbf{x}_{bc}^i), \frac{\partial u}{\partial \theta}(t_{bc}^j, \mathbf{x}_{bc}^j) \right\rangle \right], \quad (5.5)$$

$$K_r = \left[\left\langle \frac{\partial \mathcal{R}[u]}{\partial \theta}(t_r^i, \mathbf{x}_r^i), \frac{\partial \mathcal{R}[u]}{\partial \theta}(t_r^j, \mathbf{x}_r^j) \right\rangle \right], \quad (5.6)$$

where $\mathcal{R}[\cdot]$ denotes the residual operator defined in (2.4).

With this definition, we can establish an NTK-based weighting scheme as shown below

$$\hat{\lambda}_{ic} = \frac{Tr(\mathbf{K}_{ic}) + Tr(\mathbf{K}_{bc}) + Tr(\mathbf{K}_r)}{Tr(\mathbf{K}_{ic})}, \quad (5.7)$$

$$\hat{\lambda}_{bc} = \frac{Tr(\mathbf{K}_{ic}) + Tr(\mathbf{K}_{bc}) + Tr(\mathbf{K}_r)}{Tr(\mathbf{K}_{bc})}, \quad (5.8)$$

$$\hat{\lambda}_r = \frac{Tr(\mathbf{K}_{ic}) + Tr(\mathbf{K}_{bc}) + Tr(\mathbf{K}_r)}{Tr(\mathbf{K}_r)}. \quad (5.9)$$

We proceed by updating the λ_i values using a moving average, as previously described. As detailed in Appendix A, the eigenvalues of NTK characterize the convergence rate of a loss function. Higher eigenvalues imply a faster convergence rate. Given that the trace of an NTK matrix is equal to the sum of all its eigenvalues, this scheme aims to balance the convergence rates of different loss terms such that their convergence rates are comparable to one another. In practice, it should be noted that we avoid constructing the full NTK matrix. Instead, we evaluate only the diagonal elements of the NTK matrix for computing the weights, which significantly saves computational resources.

We observed that while the performance of the gradient-based and NTK-based weighting schemes is similar, the updated weights in the gradient-based scheme are less stable compared to the NTK-based scheme. This instability may be attributed to the noisy back-propagated gradients due to random mini-batches. However, the NTK-based scheme demands a higher computational cost, making it more difficult to scale to complex problems. As a result, we generally recommend employing the gradient-based scheme as a first choice.

5.3 Curriculum Training

While the techniques detailed in the preceding sections have greatly enhanced the performance and application range of PINNs, there remain certain complex domains where PINNs encounter challenges, especially in scenarios where high predictive accuracy is required. For example, when simulating chaotic dynamical systems such as the Navier-Stokes equations at high Reynolds numbers, enhanced accuracy is required to prevent error accumulation and trajectory divergence. In this section, we aim to shed light on these challenging areas and explore pathways to overcome them.

A promising approach we will delve into is the curriculum training strategy introduced by Krishnapriyan *et. al.* [48]. The core idea involves decomposing the entire optimization task for PINNs into a sequence of more manageable sub-tasks. In this work, we mainly focus on integrating this strategy into our training pipeline for solving time-dependent PDEs and singular perturbation problems.

For time-dependent PDEs, we divide the temporal domain into smaller intervals and employ Algorithm 2 to train PINNs for solving the PDE within each of these intervals. Except for the first time window, the initial condition for each subsequent time window is updated using the prediction from the last time-step of the previous time window. This approach can be viewed as a temporal domain decomposition strategy, and significantly reduces the optimization difficulty of learning the full evolution of a dynamical system while increasing computational costs due to model retraining for each window.

It is worth noting that we also partition the temporal domain in Algorithm 2 to compute the causal weights within the time-window. We emphasize that the causal weighting shares a similar motivation with “time-marching”, in the sense of respecting temporal causality by learning the solution sequentially along the time axis. Nevertheless, the causal weighting discussed in section 5.1 should not be considered a replacement for time-marching approaches, but rather a crucial enhancement, as violations of causality may still occur within each time window of a time-marching algorithm.

In addressing singular perturbation problems, our strategy hinges on a progressive approach. We initiate the training process with a less singular PDE and progressively increase its singularity throughout the training. For example, if our goal is to solve the Navier-Stokes equation at moderately high Reynolds numbers, we typically start by training a model for a lower Reynolds number and use this result as a suitable initialization for minimizing PDE residuals at higher Reynolds numbers. Through our experiments, we have observed that this approach effectively stabilizes the training process. It reduces the likelihood of PINNs becoming trapped in unfavorable local minima, thus enabling them to accurately capture complex and nonlinear PDE solutions. For a more concrete illustration, readers are directed to the example of lid-driven cavity flow in section 7.5.

6 Miscellaneous

In this section, we introduce several aspects that researchers and practitioners should consider when using PINNs to promote robust and optimal performance. The discussion highlights the importance of selecting appropriate optimizers and learning rates, imposing exact boundary conditions, employing random sampling and a modified MLP architecture.

6.1 Optimizer and learning rate

Numerous optimizers have been developed for deep learning applications; however, we find that the Adam optimizer consistently yields good performance without heavy tuning. Moreover, we do not recommend using weight decay especially for forward problems, as it tends to degrade the resulting predictive accuracy. Furthermore, the learning rate is a crucial factor in PINNs' performance. Our experience suggests that an initial learning rate of 0.001, combined with exponential decay, typically yields good results.

6.2 Random sampling

The choice of an appropriate sampling strategy for collocation points plays an important role in the training efficiency and model performance. In comparison to full batch sampling, random sampling significantly reduces the memory requirements and the computational cost of each iteration. More importantly, random sampling introduces regularization effects, which ultimately contribute to the improved generalization capabilities of PINNs. Based on our observations, training PINNs using full-batch gradient descent may result in over-fitting the PDE residuals. Consequently, we strongly recommend using random sampling in all PINN simulations to achieve optimal performance.

6.3 Imposing boundary conditions

Recent work by Dong *et al.* [64] showed how to strictly impose periodic boundary conditions in PINNs as hard-constraints, which not only effectively reduces the number of loss constraints but also significantly enhances training convergence and predictive accuracy. To illustrate the main idea, let us consider enforcing periodic boundary conditions with period P in a one-dimensional setting. Our goal is to build a network architecture satisfying

$$u^{(l)}(a) = u^{(l)}(a + P), \quad l = 0, 1, 2, \dots \quad (6.1)$$

To this end, we construct a special Fourier feature embedding of the form

$$\mathbf{v}(x) = (\cos(\omega x), \sin(\omega x)), \quad (6.2)$$

with $\omega = \frac{2\pi}{L}$. Then, for any network representation u_θ , it can be proved that any $u_\theta(v(x))$ exactly satisfies the periodic boundary condition.

The same idea can be directly extended to higher-dimensional domains. For instance, let (x, y) denote the coordinates of a point in two dimensions, and suppose that $u(x, y)$ is a smooth periodic function to be approximated in a periodic cell $[a, a + P_x] \times [b, b + P_y]$, satisfying the following constraints

$$\frac{\partial^l}{\partial x^l} u(a, y) = \frac{\partial^l}{\partial x^l} u(a + P_x, y), \quad y \in [b, b + P_y], \quad (6.3)$$

$$\frac{\partial^l}{\partial y^l} u(x, a) = \frac{\partial^l}{\partial y^l} u(x, b + P_y), \quad x \in [a, a + P_x], \quad (6.4)$$

for $l = 0, 1, 2, \dots$, where P_x and P_y are the periods in the x and y directions, respectively. Similar to the one-dimensional setting, these constraints can be implicitly encoded in a neural network by constructing a two-dimensional Fourier features embedding as

$$\mathbf{v}(x, y) = [\cos(\omega_x x), \sin(\omega_x x), \cos(\omega_y y), \sin(\omega_y y)] \quad (6.5)$$

with $w_x = \frac{2\pi}{P_x}, w_y = \frac{2\pi}{P_y}$.

For time-dependent problems, we simply concatenate the time coordinates t with the constructed Fourier features embedding, i.e., $u_\theta([t, \mathbf{v}(x)])$, or $u_\theta([t, \mathbf{v}(x, y)])$. In particular, if the PDE solutions are known to exhibit periodic behavior over time, we can also enforce periodicity along the time axis. More precisely, we consider the following special Fourier embedding

$$\mathbf{w}(t, x) = [\cos(\omega_t t), \sin(\omega_t t), \mathbf{v}(t, x)] \quad (6.6)$$

where $\omega_t = \frac{2\pi}{P_t}$. The key difference is that P_t is a trainable parameter. Typically, P_t is initialized to the temporal domain's length, allowing networks to learn the solution's correct period. It is worth emphasizing that this assumption of time periodicity is not a severe restriction, and this technique can be applied to arbitrary dynamical systems, even if the solution is not periodic. This is because one can always set the initial P_t greater than the length of the temporal domain.

Lastly, other types of boundary conditions, including Dirichlet, Neumann, Robin, etc., can also be enforced in a "hard" manner by modifying the network outputs, see [65, 66] for more details.

6.4 Modified MLP

In practice, we found that a simple modification of MLPs proposed by Wang *et al.* [26] demonstrates an enhanced capability for learning nonlinear and complex PDE solutions. The forward pass of an L -layer modified MLP is defined as follows. First, we introduce two encoders for the input coordinates

$$\mathbf{U} = \sigma(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1), \quad \mathbf{V} = \sigma(\mathbf{W}_2 \mathbf{x} + \mathbf{b}_2). \quad (6.7)$$

Then, for $l = 1, 2, \dots, L$,

$$\mathbf{f}^{(l)}(\mathbf{x}) = \mathbf{W}^{(l)} \cdot \mathbf{g}^{(l-1)}(\mathbf{x}) + \mathbf{b}^{(l)}, \quad (6.8)$$

$$\mathbf{g}^{(l)}(\mathbf{x}) = \sigma(\mathbf{f}_\theta^{(l)}(\mathbf{x})) \odot \mathbf{U} + (1 - \sigma(\mathbf{f}_\theta^{(l)}(\mathbf{x}))) \odot \mathbf{V}. \quad (6.9)$$

The final network output is given by

$$\mathbf{f}_\theta(\mathbf{x}) = \mathbf{W}^{(L+1)} \cdot \mathbf{g}^{(L)}(\mathbf{x}) + \mathbf{b}^{(L+1)}. \quad (6.10)$$

Here, σ denotes a nonlinear activation function, and $\odot$ denotes an element-wise multiplication. All trainable parameters are given by

$$\theta = \{\mathbf{W}_1, \mathbf{b}_1, \mathbf{W}_2, \mathbf{b}_2, (\mathbf{W}^{(l)}, \mathbf{b}^{(l)})_{l=1}^{L+1}\}. \quad (6.11)$$

This architecture is almost the same as a standard MLP network, with the addition of two encoders and a minor modification in the forward pass. Specifically, the inputs $\mathbf{x}$ are embedded into a feature space via two encoders $\mathbf{U}, \mathbf{V}$, respectively, and merged in each hidden layer of a standard MLP using a point-wise multiplication. In our experience, the modified MLP demands greater computational resources; however, it generally outperforms the standard MLP in effectively minimizing PDE residuals, thereby yielding more accurate results.

7 Results

In this section, we present a series of challenging benchmarks for evaluating PINNs performance and illustrate the effectiveness of Algorithm 1, along with the proposed training strategies. Besides, we showcase the state-of-the-art results for each benchmark, demonstrating the current performance capacity of PINNs. More importantly, we believe that these results can establish robust and strong baselines, enabling future researchers to perform thorough evaluations and comparisons of their novel methods. This paves the way for continued innovation and developments in this field.

For each benchmark, except for the last two, we perform comprehensive ablation studies to assess the effectiveness of the methods presented in the previous sections. In each ablation study we systematically disable each methodological component individually, while keeping the others active under the same hyper-parameter settings, and evaluate the resulting relative L^2 error and run-time. This allows us to isolate the effects of each component and understand their contribution to the overall model performance. Throughout all ablation studies, we maintain the following hyper-parameter settings, unless stated otherwise. Specifically, we employ an MLP with 4 hidden layers, 256 neurons per hidden layer, and tanh activation functions as our backbone, initializing it using the Glorot scheme [55]. For model training, we use the Adam optimizer [67], starting with a learning rate of 10^{-3} and an exponential decay with a decay

Benchmark	Relative L^2 error
Allen-Cahn equation	5.37×10^{-5}
Advection equation	6.88×10^{-4}
Stokes flow	8.04×10^{-5}
Kuramoto–Sivashinsky equation	1.61×10^{-1}
Lid-driven cavity flow (Re=3200)	1.58×10^{-1}
Navier–Stokes flow in a torus	2.45×10^{-1}
Navier–Stokes flow around a cylinder	–

Table 1: State-of-the-art relative L^2 error for various benchmark equations using our proposed model.

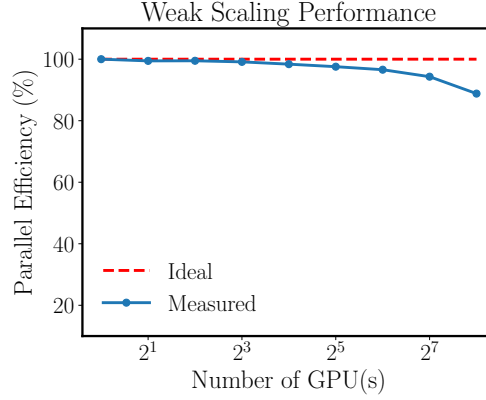


Figure 2: Efficiency of weak scaling using the Navier-Stokes flow (section 7.6) as a benchmark. We employ a neural network with hyper-parameters shown in Table 12 and measure the execution time for 10,000 iterations, maintaining a consistent batch size of 40960 per GPU.

rate of 0.9 for every 2,000 decay steps. The collocation points are uniformly sampled from the computational domain with a batch size of 4096. The total number of training iterations can vary depending on the complexity of the example.

Furthermore, we conduct extensive hyper-parameter sweeps across various learning rate schedules, network sizes, and activations, in order to produce state-of-the-art results for each example. Note that the hyper-parameter settings for our ablation studies differ from those yielding the best results. We summarize our results in Table 1 and provide detailed hyper-parameter settings for our optimal models in the Appendix. Throughout all numerical experiments, when applicable, we enforce the exact periodic boundary conditions as described in section 6.

The code and data accompanying this manuscript will be made publicly available at <https://github.com/PredictiveIntelligenceLab/jaxpi>. It should be highlighted that our implementation automatically supports efficient data-parallel multi-GPU training. As illustrated in Figure 2, we show great weak scaling capabilities up to 256 GPUs, enabling the effective simulation of large-scale problems. Additionally, our code includes valuable utilities for monitoring gradient norms and NTK eigenvalues throughout training—key metrics essential for identifying potential training issues.

7.1 Allen-Cahn equation

We start with 1D Allen-Cahn equation, a representative case with which conventional PINN models are known to struggle. It takes the form

$$u_t - 0.0001u_{xx} + 5u^3 - 5u = 0, \quad t \in [0, 1], x \in [-1, 1], \quad (7.1)$$

$$u(0, x) = x^2 \cos(\pi x), \quad (7.2)$$

$$u(t, -1) = u(t, 1), \quad (7.3)$$

$$u_x(t, -1) = u_x(t, 1). \quad (7.4)$$

For this example, we first train a conventional PINN model to diagnose potential issues. In Figure 3, we visualize the histogram of back-propagated gradients, the resulting NTK eigenvalues and the temporal PDE residual loss (equation

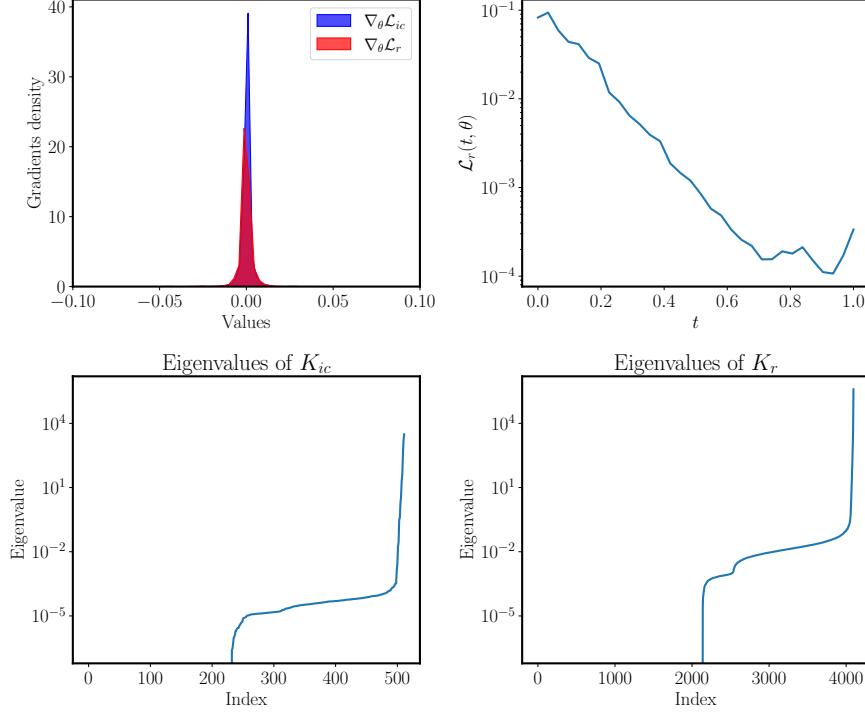


Figure 3: *Allen Cahn equation*: Analysis of training a plain PINN model for 10,000 iterations. *Top left*: Histograms of back-propagated gradients of the PDE residual loss and initial condition loss at the last iteration. *Top right*: Temporal PDE residual loss at the last iteration. *Bottom*: NTK eigenvalues of K_{ic} and K_r at the last iteration.

(2.10)) at the early stages of training. On the top left panel, one can see that the gradients of PDE residual loss dominates those of the initial condition loss, which implies unbalanced back-propagated gradients. Moreover, the top right panel reveals that the network tends to minimize the PDE residuals at later times first, suggesting a violation of causality. In the bottom panel, a rapid decay in the NTK eigenvalues can be observed, indicating the presence of spectral bias. These findings strongly suggest that conventional PINNs suffer from multiple severe training pathologies, which need to be addressed simultaneously to yield satisfactory results.

To showcase the effectiveness of the proposed training pipeline in addressing these issues, we employ Algorithm 1 and disable individual methodological components one-at-a-time. The results are summarized in Table 2 and Figure 4. It can be concluded that the full algorithm yields the best performance in terms of relative L^2 error of 5.84×10^{-4} . Removing any individual component from the algorithm generally leads to a worse performance, which indicates the positive contributions of each component to the overall model performance. The most significant negative impact on performance occurs when disabling the Fourier Feature embedding, resulting in a relative L^2 error of 4.35×10^{-1} . It implies that the spectral bias degrades the predictive accuracy the most for this example. Furthermore, it is worth noting that the run-times across different configurations are relatively similar, except for the case corresponding to conventional PINNs, which shows a slightly shorter run-time of 12.93 minutes. This highly suggests the computational efficiency of each component presented in Algorithm 1. Finally, we present our best result in Figure 5, whereas Table 6 details the corresponding hyper-parameter configuration, and Figure 19 visualizes the loss convergence and the weight changes during training. One can see that the predicted solution achieves an excellent agreement with the reference solution, yielding a relative L^2 error of 5.37×10^{-5} .

7.2 Advection equation

Our second example is a 1D advection equation; a linear hyperbolic equation commonly used to model transport phenomena. It takes the form

$$\frac{\partial u}{\partial t} + c \frac{\partial u}{\partial x} = 0, \quad t \in [0, 1], \quad x \in (0, 2\pi), \quad (7.5)$$

$$u(0, x) = g(x), \quad x \in (0, 2\pi), \quad (7.6)$$

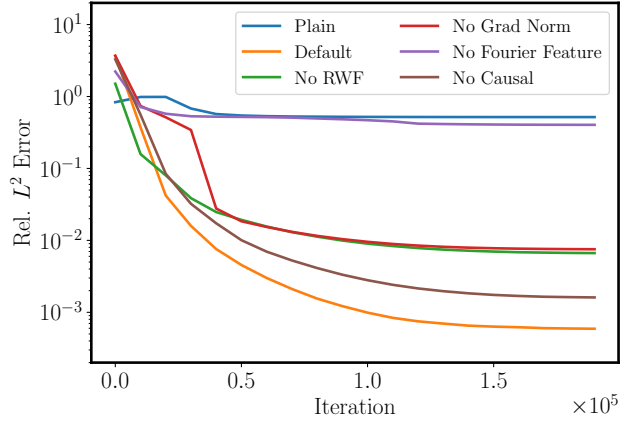


Figure 4: *Allen Cahn equation*: Convergence of relative L^2 error for the ablation study with different components disabled. Plain: Conventional PINN formulation. Default: PINN model trained using Algorithm 1. No RWF: PINN model trained using Algorithm 1 without random weight factorization. No Grad Norm: PINN model trained using Algorithm 1 without grad norm weighting scheme. No Fourier feature: PINN model trained using Algorithm 1 without random Fourier feature embeddings. No Causal: PINN model trained using Algorithm 1 without casual weighting.

Ablation Settings				Performance	
Fourier Feature	RWF	Grad Norm	Causal	Rel. L^2 error	Run time (min)
✓	✓	✓	✓	5.84×10^{-4}	16.26
✗	✓	✓	✓	4.35×10^{-1}	13.20
✓	✗	✓	✓	6.62×10^{-3}	16.53
✓	✓	✗	✓	7.51×10^{-3}	16.36
✓	✓	✓	✗	1.59×10^{-3}	16.11
✗	✗	✗	✗	51.74×10^{-1}	12.93

Table 2: *Allen Cahn equation*: Relative L^2 error and run time for an ablation study illustrating the impact of disabling individual components of the proposed training pipeline. Note that the GPU run time may vary due to factors such as hardware utilization, batch processing, and other computational loads.

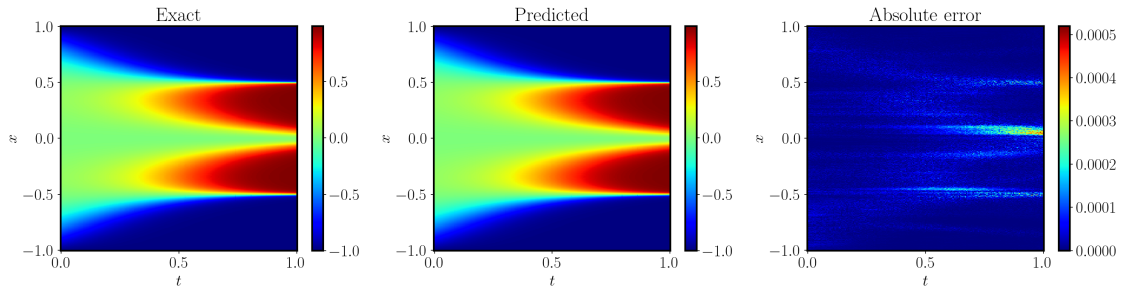


Figure 5: *Allen Cahn equation*: Comparison of the best prediction against the reference solution. The resulting relative L^2 error is 5.37×10^{-5} . The hyper-parameter configuration can be found in Table 6.

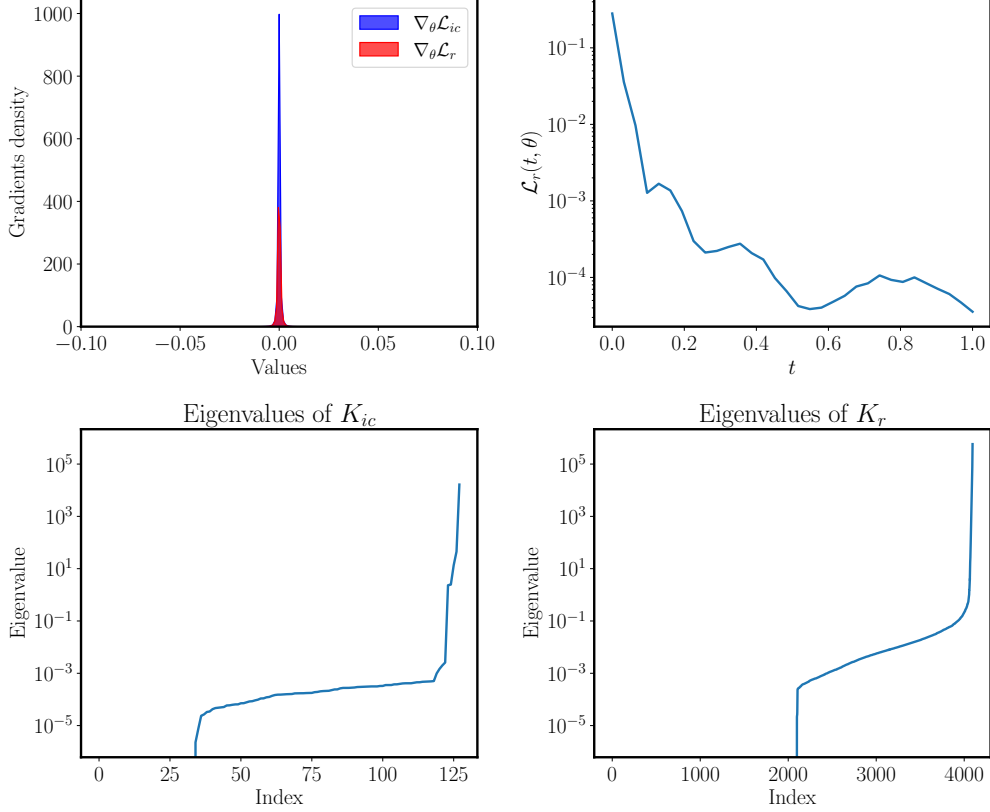


Figure 6: *Advection equation*: Analysis of training a plain physics-informed neural network for 10,000 iterations. *Top left*: Histograms of back-propagated gradients of the PDE residual loss and initial condition loss at the last iteration. *Top right*: Temporal PDE residual loss at the last iteration. *Bottom*: NTK eigenvalues of K_{ic} and K_r at the last iteration.

with periodic boundary conditions. This example has been studied in [48, 31], exposing some of the limitations that PINNs suffer from as the transport velocity c is increased. In our experiments, we consider the challenging setting of $c = 80$ with an initial condition $g(x) = \sin(x)$.

Analogous to the first example, we train a conventional PINN model with the aim of identifying the issues that lead to inaccurate results. As illustrated in Figure 3, it is evident that PINNs experience the same challenges as those observed in the first example. This observation strongly suggests the widespread nature of these issues in the training of PINNs, further emphasizing the necessity of addressing them to obtain robust and accurate PINN models.

As mentioned in section 6.3, we can impose the spatial and temporal periodicity by

$$\mathbf{v}(t, x) = [\cos(\omega_t t), \sin(\omega_t t), \cos(\omega_x x), \sin(\omega_x x)], \quad (7.7)$$

where $\omega_t = \frac{2\pi}{P_t}$ and $\omega_x = \frac{2\pi}{P_x}$ with $P_x = 2\pi$ and P_t a trainable parameter. For this example, we incorporate the imposition of temporal periodicity in Algorithm 1 and subsequently perform an ablation study on the enhanced algorithm. The performance of various configurations is summarized in Table 3. One can conclude that the integration of all the techniques together yields the optimal accuracy. The exclusion of any of these elements, especially the time periodicity, Fourier Features and the grad norm weighting scheme, leads to a significant increase in test error, highlighting their crucial role in achieving accurate results. Additionally, we present the state-of-the-art result in Figure 8. We see that the model prediction achieves an excellent agreement with the exact solution, with an relative L^2 error of $6.88 \cdot 10^{-4}$. The hyper-parameter configuration and loss convergence are presented in Table 7 and Figure 20, respectively.

7.3 Stokes flow

In this example, we explore a specific example of Stokes flow with the aim of emphasizing the importance of non-dimensionalization in PINNs training. Stokes flow is a fluid flow regime where viscous forces outweigh inertial forces, occurring in scenarios such as small particle motion in liquids, fluid flow through porous media, and microorganism

Ablation Settings					Performance	
Time Period	Fourier Feature	RWF	Grad Norm	Causal	Rel. L^2 error	Run time (min)
✓	✓	✓	✓	✓	1.02×10^{-2}	9.18
✗	✓	✓	✓	✓	7.37×10^{-1}	8.76
✓	✗	✓	✓	✓	4.29×10^{-1}	7.60
✓	✓	✗	✓	✓	1.31×10^{-2}	9.25
✓	✓	✓	✗	✓	1.13×10^0	7.46
✓	✓	✓	✓	✗	1.49×10^{-2}	9.18
✗	✗	✗	✗	✗	9.51×10^{-1}	7.12

Table 3: *Advection equation*: Relative L^2 error and run time for an ablation study illustrating the impact of disabling individual components of the proposed technique and training pipeline.

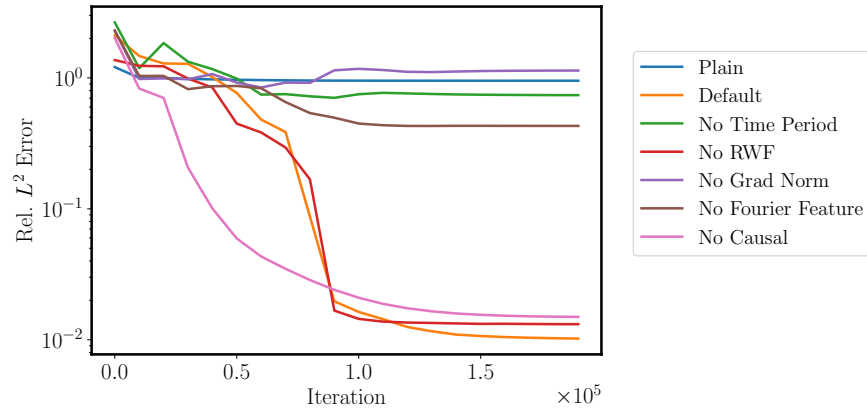


Figure 7: *Advection equation*: Convergence of relative L^2 error for the ablation study with different components disabled. Plain: Conventional PINN formulation. Default: PINN model trained imposing time periodicity and using Algorithm 1. No Time Period: PINN model trained using Algorithm 1. No RWF: PINN model trained imposing time periodicity and using Algorithm 1 without random weight factorization. No Grad Norm: PINN model trained imposing time periodicity and using Algorithm 1 without grad norm weighting scheme. No Fourier feature: PINN model trained imposing time periodicity and using Algorithm 1 without random Fourier feature embeddings. No Causal: PINN model trained imposing time periodicity and using Algorithm 1 without casual weighting.

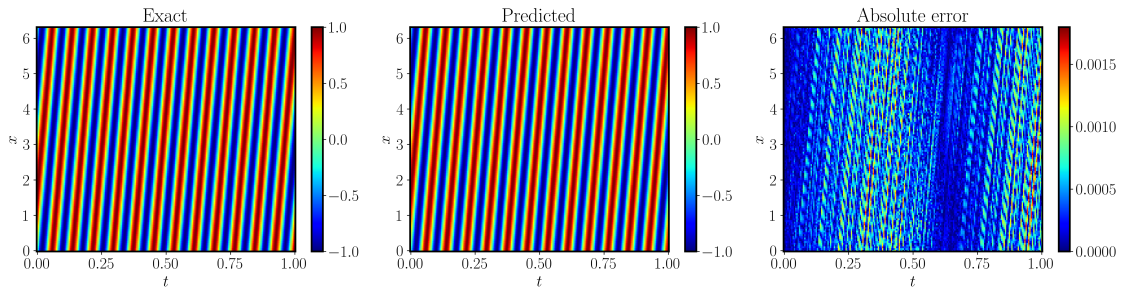


Figure 8: *Advection equation*: Comparison of the best prediction against the reference solution obtained from the hyper-parameter sweep. The resulting relative L^2 error is 6.88×10^{-4} . The hyper-parameter configuration can be found in Table 7.

locomotion in fluid environments. The governing equation is given by

$$-\nu \Delta \mathbf{u} + \nabla p = 0, \quad (7.8)$$

$$\nabla \cdot \mathbf{u} = 0, \quad (7.9)$$

where $\mathbf{u} = (u, v)$ defines the velocity and p the pressure, and ν is the kinematic viscosity.

As depicted in Figure 9, the underlying geometry is a pipe $\Omega = [0, 2.2] \times [0, 0.41] \setminus B_r(0.2, 0.2)$ with a circular cylinder obstacle of radius $r = 0.05$. For the top and bottom walls $\Gamma_1 = [0, 2.2] \times 0.41$ and $\Gamma_2 = [0, 2.2] \times 0$ as well as the boundary $S = \partial B_r(0.2, 0.2)$, we impose the no-slip boundary condition

$$u|_{\Gamma_1} = u|_{\Gamma_2} = u|_S = 0. \quad (7.10)$$

At the inlet $\Gamma_3 = 0 \times [0, 0.41]$, a parabolic inflow profile is prescribed,

$$\mathbf{u}(0, y) = \mathbf{u}_{\text{in}} = \left(\frac{4Uy(0.41 - y)}{0.41^2}, 0 \right), \quad (7.11)$$

with a maximum velocity $U = 0.3$. At the outlet $\Gamma_4 = 2.2 \times [0, 0.41]$, we define the outflow condition

$$\nu \partial_{\mathbf{n}} \mathbf{u} - p \mathbf{n} = 0, \quad (7.12)$$

where $\mathbf{n}$ denotes the outer normal vector.

To non-dimensionalize the system, we select the characteristic flow velocity and length as $U^* = 0.2$ and $L^* = 0.1$, respectively. This results in a Reynolds number of

$$\text{Re} = \frac{U^* L^*}{\nu} = \frac{0.2 \cdot 0.1}{0.001} = 20. \quad (7.13)$$

We can then define the non-dimensionalized variables as

$$\mathbf{x}^* = \frac{\mathbf{x}}{L^*}, \quad \mathbf{u}^* = \frac{\mathbf{u}}{U^*}, \quad p^* = \frac{p L^*}{\nu U^*}, \quad \nabla^* = L^* \nabla. \quad (7.14)$$

By substituting these scales into the dimensionalized system, we obtain the non-dimensionalized PDE as

$$-\frac{1}{\text{Re}} \Delta \mathbf{u}^* + \nabla^* p^* = \mathbf{0} \quad \text{in } \Omega^*, \quad (7.15)$$

$$\nabla^* \cdot \mathbf{u}^* = 0 \quad \text{in } \Omega^*, \quad (7.16)$$

$$\mathbf{u}^* = 0 \quad \text{on } \Gamma_1^* \cup \Gamma_2^* \cup S^*, \quad (7.17)$$

$$\mathbf{u}^* = \frac{\mathbf{u}_{\text{in}}}{U^*} \quad \text{on } \Gamma_3^*, \quad (7.18)$$

$$\frac{1}{\text{Re}} \frac{\partial \mathbf{u}^*}{\partial \mathbf{n}} - p^* \mathbf{n} = 0 \quad \text{on } \Gamma_4^*, \quad (7.19)$$

where Ω^* , S^* and $\{\Gamma_i\}_{i=1}^4$ denote the non-dimensionalized domains, respectively.

To perform an ablation study for Algorithm 1, we employ an MLP with 4 hidden layers, 128 neurons per hidden layer, and GeLU activation functions and train each model for 10^5 iterations of gradient descent using the Adam optimizer. The results are summarized in Table 4, and strongly indicate the positive impact of all proposed components on model performance; disabling any one component leads to worse predictive accuracy. In particular, comparing the performance of the configurations with non-dimensionalization enabled (1st row) to the ones with non-dimensionalization disabled (5th rows), we observe a substantial increase in the relative L^2 error when non-dimensionalization is removed. This observation highlights the importance of non-dimensionalization in achieving optimal performance for solving the Stokes equation. Moreover, as evidenced by the 3rd and 4th rows of the table, models trained without Fourier features and RWF fail to capture the correct solution, thus implying their essential contribution to the overall model performance. Lastly, we present the results of a fine-tuned PINN model in Figure 11, which exhibits excellent agreement with the reference solution and achieves a relative L^2 error of 8.04×10^{-5} . The detailed hyper-parameter configuration and the loss convergence are respectively shown in Table 8 and Figure 21.

7.4 Kuramoto–Sivashinsky equation

In this example, we aim to demonstrate the potential of PINNs in simulating chaotic dynamics and highlight the necessity of adopting a time-marching strategy in scenarios where high predictive accuracy is needed. To this end, we

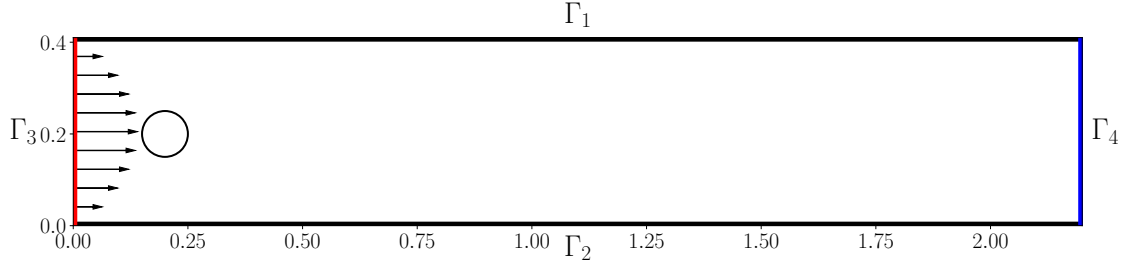


Figure 9: *Stokes equation*: Illustration of the pipe geometry for Stokes flow.

Ablation Settings				Performance	
Fourier Feature	RWF	Grad Norm	Non-dimensionalization	Rel. L^2 error	Run time (min)
✓	✓	✓	✓	5.41×10^{-4}	9.51
✗	✓	✓	✓	9.56×10^{-1}	7.93
✓	✗	✓	✓	9.86×10^{-1}	9.58
✓	✓	✗	✓	1.01×10^{-2}	8.63
✓	✓	✓	✗	9.74×10^{-1}	9.58
✗	✗	✗	✗	9.21×10^{-1}	7.95

Table 4: *Stokes equation*: Relative L^2 error and run time for an ablation study illustrating the impact of disabling non-dimensionalization and individual components of the proposed training pipeline. The error is measured against the norm of flow velocity $\|\mathbf{u}\|_2 = \sqrt{u^2 + v^2}$.

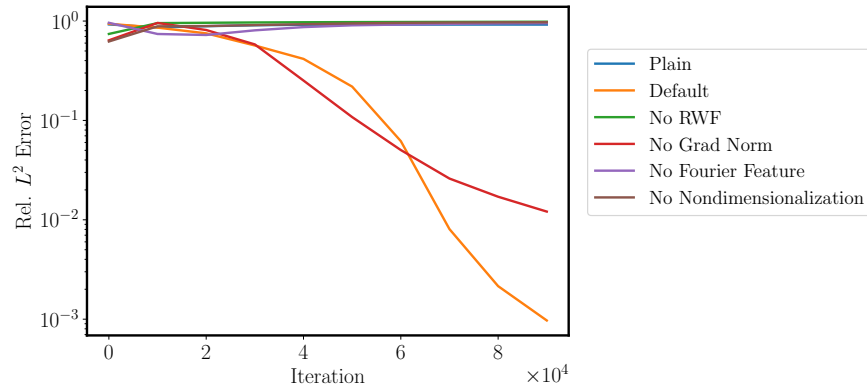


Figure 10: *Stokes equation*: Convergence of relative L^2 error for the ablation study with different components disabled.

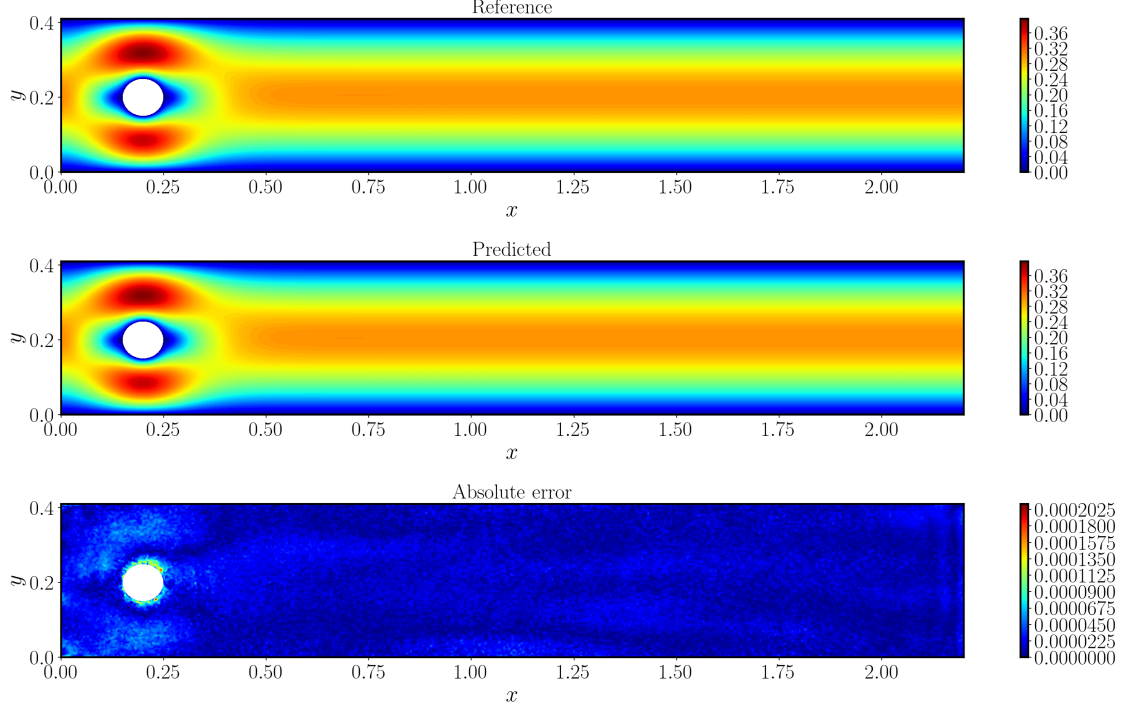


Figure 11: *Stokes equation*: Comparison of the best prediction against the reference solution obtained from the hyper-parameter sweep. The resulting relative L^2 error is 8.04×10^{-5} . The hyper-parameter configuration can be found in Table 8.

consider the Kuramoto–Sivashinsky equation, which exhibits a wealth of spatially and temporally nontrivial dynamical behavior, and has served as a model example in efforts to understand and predict the complex dynamical behavior associated with a variety of physical systems. The equation takes the form

$$u_t + \alpha uu_x + \beta u_{xx} + \gamma u_{xxxx} = 0, \quad t \in [0, 1], \quad x \in [0, 2\pi], \quad (7.20)$$

subject to periodic boundary conditions and an initial condition

$$u(0, x) = u_0(x). \quad (7.21)$$

Specifically, we take $\alpha = 100/16, \beta = 100/16^2, \gamma = 100/16^4$ and $u_0(x) = \cos(x)(1 + \sin(x))$.

Based on our experience, it appears highly challenging to conduct long-term integration of this PDE system via a single-shot training of PINNs. This could potentially be attributed to the inherently chaotic nature of the system and the insufficient accuracy of PINNs predictions. To illustrate this point, we train a PINN to simulate the dynamical system up to different final time T without time-marching, while keeping the same hyper-parameter settings. As shown in the left panel of Figure 12, we can see that the resulting relative L^2 error drastically increases for larger temporal domains and eventually leads to a failure in correctly capturing the PDE solution. This illustrates the necessity for applying time-marching in order to mitigate the difficulties of approximation and optimization, thus leading to more accurate results. However, we must emphasize that the computational cost of time-marching is considerably larger than one-shot learning as one needs to train multiple PINN models sequentially. It would be interesting to explore the acceleration of this training process in the future work.

Moreover, we present an ablation study on Algorithm 1 and summarize our results in Table 5. It can be concluded that all proposed components positively contribute to the overall model performance, and removing of any one of them results in increased errors. Notably, the use of modified MLP greatly enhances the predictive accuracy, reflected in the substantial error reduction from 2.98×10^{-3} to 1.42×10^{-4} . From our experience, modified MLPs typically outperforms plain MLPs, especially for tackling non-linear PDE systems. Furthermore, the predicted solution obtained from our best model is visualized in Figure 13, which is in a good agreement with the ground truth. Nevertheless, some discrepancies can be observed near $t = 1$, which may be due to the error accumulation and the inherent nature of chaos. More details of implementation and training are provided in Table 9 and Figure 22.

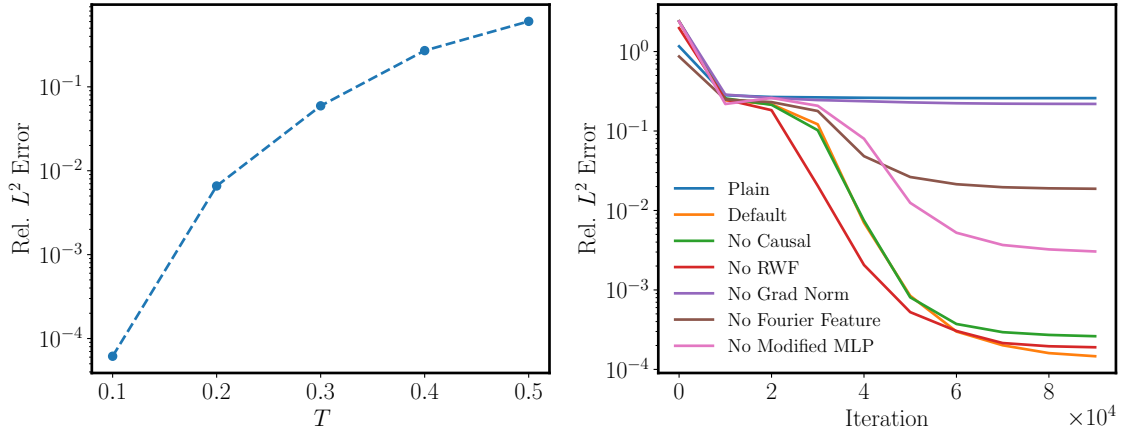


Figure 12: *Kuramoto–Sivashinsky equation*: Left: Relative L^2 errors from one-shot PINN training for different system final time T under the same hyper-parameter setting. Right: Convergence of relative L^2 error for the ablation study with different components disabled.

Ablation Settings					Performance	
Modified MLP	Fourier Feature	RWF	Grad Norm	Causal	Rel. L^2 error	Run time (min)
✓	✓	✓	✓	✓	1.42×10^{-4}	13.33
✗	✓	✓	✓	✓	2.98×10^{-3}	6.21
✓	✗	✓	✓	✓	1.86×10^{-2}	7.60
✓	✓	✗	✓	✓	1.86×10^{-4}	14.11
✓	✓	✓	✗	✓	2.19×10^{-1}	14.11
✓	✓	✓	✓	✗	2.58×10^{-4}	9.18
✗	✗	✗	✗	✗	2.59×10^{-1}	7.12

Table 5: *Kuramoto–Sivashinsky equation*: Relative L^2 error and run time for an ablation study illustrating the impact of disabling individual components of the proposed technique and training pipeline.

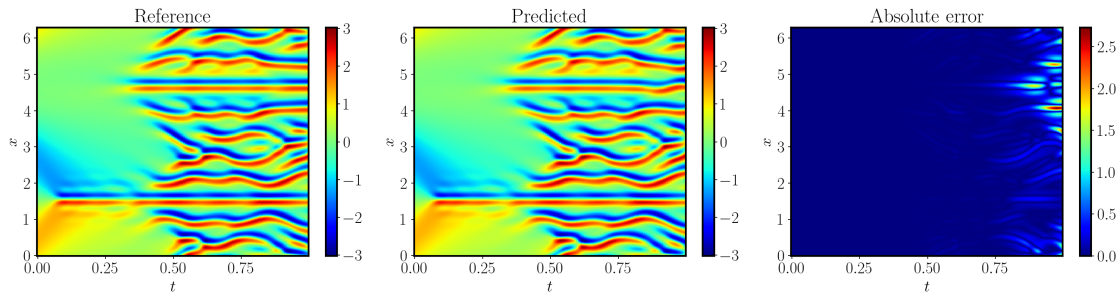


Figure 13: *Kuramoto–Sivashinsky equation*: Comparison of the best prediction against the reference solution. The relative L^2 error of the spatial temporal predicted solution is 1.61×10^{-1} . Note that the the majority of this error is attributed to last few time steps.

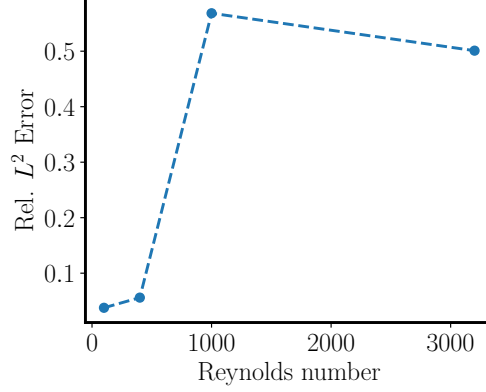


Figure 14: *Lid-driven cavity*: Relative L^2 error of training PINNs with Algorithm 1 at different Reynolds numbers $Re \in [100, 400, 1000, 3200]$.

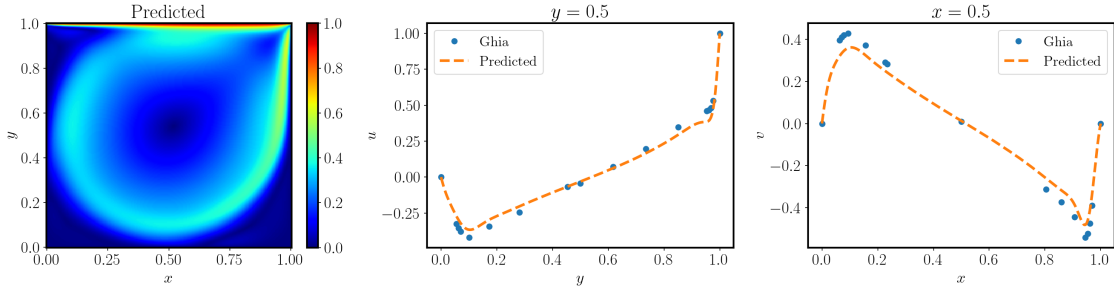


Figure 15: *Lid-driven cavity* ($Re=3200$): *Left*: Predicted velocity of the fine-tuned model. *Right*: Comparison of the predicted velocity profiles on the vertical and horizontal center-lines against Ghia *et al.* [68]. The resulting relative L^2 error against the reference solution is 1.58×10^{-1} .

7.5 Lid-driven cavity flow

In this example, we consider a classical benchmark problem in computational fluid dynamics, describing the motion of an incompressible fluid in a two-dimensional square cavity. The system is governed by the incompressible Navier–Stokes equations written in a non-dimensional form

$$\mathbf{u} \cdot \nabla \mathbf{u} + \nabla p - \frac{1}{Re} \Delta \mathbf{u} = 0, \quad (x, y) \in (0, 1)^2, \quad (7.22)$$

$$\nabla \cdot \mathbf{u} = 0, \quad (x, y) \in (0, 1)^2, \quad (7.23)$$

where $\mathbf{u} = (u, v)$ denotes the velocity in x and y directions, respectively, and p is the scalar pressure field. We assume $\mathbf{u} = (1, 0)$ on the top lid of the cavity, and a non-slip boundary condition on the other three walls. We are interested in the velocity and pressure distribution for a Reynolds number of 3200.

In our experience, when trained directly at a high Reynolds number, PINNs tend to be unstable and susceptible of converging to erroneous solutions. This observation is verified by the left panel of Figure 14, where we plot the relative L^2 errors from training PINNs with Algorithm 1 at varying Reynolds numbers under the same hyper-parameter settings. Our results demonstrate that PINNs struggle to yield accurate solutions for Reynolds numbers greater than 1,000. To improve this result, one effective approach is to start the training of PINNs with a lower initial Reynolds number, and gradually increase the Reynolds numbers during training. By this way, the model parameters obtained from the training with smaller Reynolds numbers serve as a good initialization when training for higher Reynolds numbers. To demonstrate this, we select an increasing sequence of Reynolds numbers $[100, 400, 1000, 3200]$ and train PINNs with Algorithm 1 for $5 \times 10^4, 5 \times 10^4, 1 \times 10^5, 5 \times 10^5$ iterations, respectively. The detailed hyper-parameter configuration is summarized in Table 10. As shown in Figure 15, our predicted velocity field agrees well with the reference results of Ghia *et al.* [68], yielding a relative L^2 error of 1.58×10^{-1} against the reference solution.

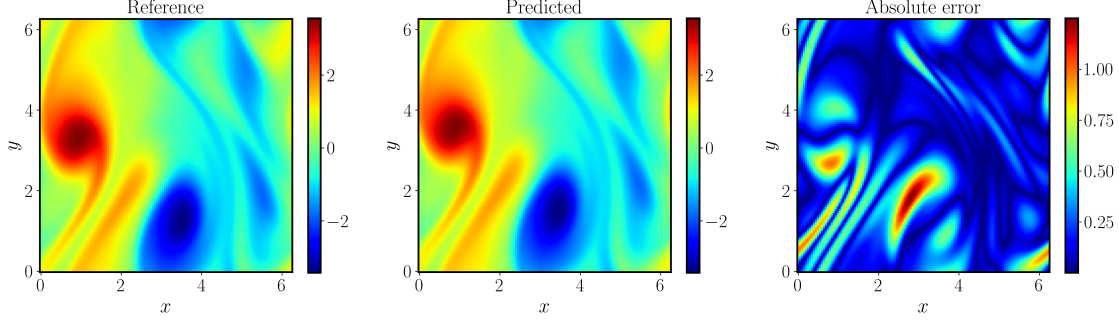


Figure 16: *Navier-Stokes flow in a torus*: Comparison of the best prediction against the reference solution at the last time step. The animation is provided in <https://github.com/PredictiveIntelligenceLab/jaxpi>.

7.6 Navier–Stokes flow in a torus

As the second to last example, our goal is to showcase the capability of PINNs in simulating incompressible Navier–Stokes flow using the velocity-vorticity formulation. The equation is given by

$$w_t + \mathbf{u} \cdot \nabla w = \frac{1}{\text{Re}} \Delta w, \quad \text{in } [0, T] \times \Omega, \quad (7.24)$$

$$\nabla \cdot \mathbf{u} = 0, \quad \text{in } [0, T] \times \Omega, \quad (7.25)$$

$$w(0, x, y) = w_0(x, y), \quad \text{in } \Omega. \quad (7.26)$$

Here, $\mathbf{u} = (u, v)$ represents the flow velocity field, $w = \nabla \times \mathbf{u}$ denotes the vorticity, and Re denotes the Reynolds number. For this example, we define $\Omega = [0, 2\pi]^2$ and set Re as 100.

As the validation and effectiveness of the proposed PINN algorithm have been rigorously proven in prior examples, our focus is on simulating the vorticity evolution up to $T = 10$ using PINNs. To this end, we split the temporal domain into 5 intervals and employ a time-marching strategy. For each interval, we use a PINN model with a modified MLP (4 hidden layers, 256 neurons per hidden layer, Tanh activations) and train it using Algorithm 1 for 10^5 iterations of gradient descent with Adam optimizer. The results of this simulation are summarized in Figure 16, which provides a visual comparison of the reference and predicted vorticity at $T = 10$. While a slight misalignment between the two can be observed, the model prediction is in good agreement with the corresponding numerical estimations. This demonstrates the capability of PINNs to closely match the reference solution, emphasizing its effectiveness in simulating vortical fluid flows.

7.7 Navier–Stokes flow around a cylinder

In our last example, we investigate a classical benchmark in computational fluid dynamics, describing the behaviour of a transient fluid in a pipe with a circular obstacle. Previous research by Chuang *et al.* [69] reported that PINNs act as a steady-flow solver, and fail to capture the phenomenon of vortex shedding. Here we challenge these findings and demonstrate that, if properly used, PINNs can successfully simulate the development of vortex shedding in this scenario.

Specifically, we consider a fluid with a density of $\rho = 1.0$ and describe its behavior using the time-dependent incompressible Navier-Stokes equations

$$\mathbf{u}_t + \mathbf{u} \cdot \nabla \mathbf{u} + \nabla p - \nu \nabla^2 \mathbf{u} = 0, \quad (7.27)$$

$$\nabla \cdot \mathbf{u} = 0, \quad (7.28)$$

with $\mathbf{u} = (u, v)$ defining the velocity field and p the pressure. The kinematic viscosity is taken as $\nu = 0.001$.

The underlying geometry is identical to Figure 9 and the boundary conditions are the same as the Stokes flow example discussed in section 7.3. However, we introduce a parabolic inflow profile with a maximum velocity of $U = 1.5$. As a result, we have characteristic flow velocity and length values of $U = 1.0$ and $L = 0.1$, respectively, and a Reynolds number of $Re = 100$.

We begin by normalizing the PDE system as follows:

$$\mathbf{x}^* = \frac{\mathbf{x}}{L^*}, \quad t^* = \frac{L^*}{U^*}, \quad \mathbf{u}^* = \frac{\mathbf{u}}{U^*}, \quad p^* = \frac{p L^*}{\nu U^*}, \quad \nabla^* = L^* \nabla. \quad (7.29)$$

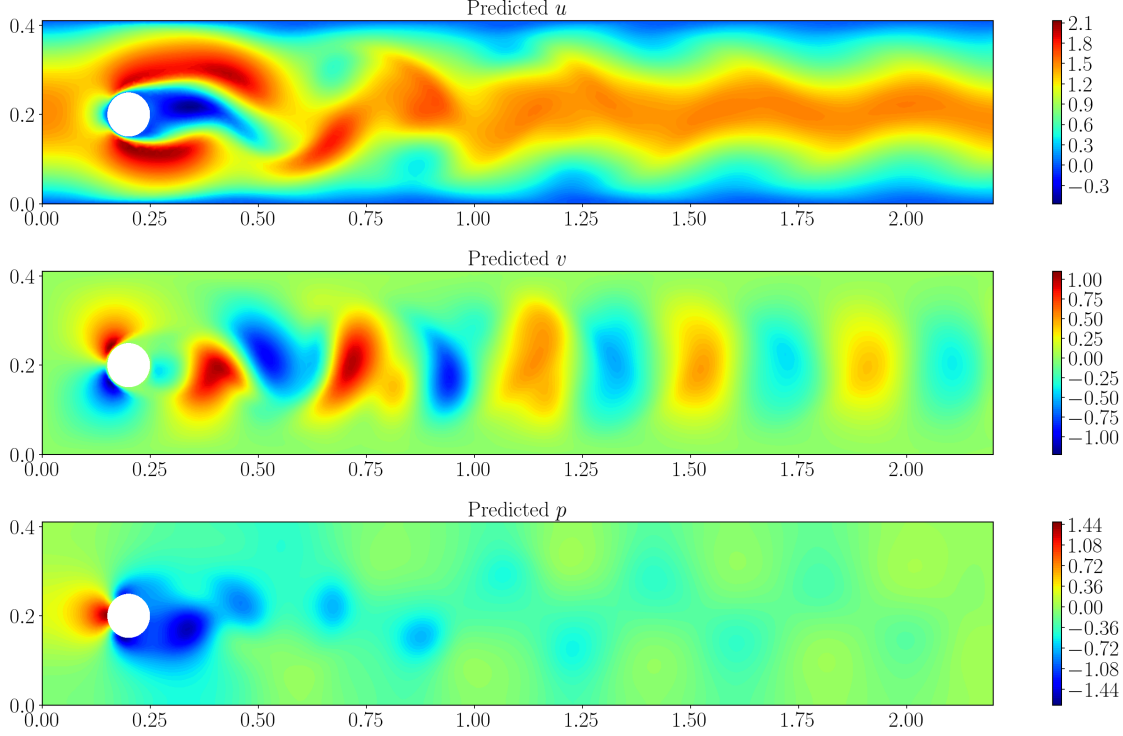


Figure 17: *Navier-Stokes flow around cylinder*: Predicted velocity field and pressure at $T = 1$. the last time step. The animation is provided in <https://github.com/PredictiveIntelligenceLab/jaxpi>.

This leads us to the non-dimensionalized equations:

$$\mathbf{u}_t^* + \mathbf{u}^* \nabla^* \mathbf{u} + \nabla p^* - \frac{1}{Re} \mathbf{u}^* = 0, \quad (7.30)$$

$$\nabla^* \cdot \mathbf{u}^* = 0. \quad (7.31)$$

To obtain a proper initial condition for PINNs, we start with a zero solution and run a numerical simulation for 4 seconds at a very coarse spatial and temporal resolution. We then use the last time-step as our initial condition for the PINNs simulation.

Using a time-marching strategy, we partition the temporal domain $[0, 10]$ into 10 individual time windows. For each window, a modified MLP is employed as our model backbone. PINN training runs for 2×10^5 iterations per window following Algorithm 1. Key hyper-parameters are detailed in Table 13. It deserves mentioning that there are more than 10 terms in the total loss and thus it is practically infeasible to manually adjust the weights of each loss. The predicted velocity and pressure field at $T = 10$ are plotted in Figure 17. For this benchmark, we do not report the test error against the numerical solution, as the start time of vortex shedding in numerical solvers fluctuates based on the underlying discretizations. To the best of our knowledge, our work presents the first empirical evidence of a PINN model being able to capture the phenomenon of vortex shedding. This finding opens up new avenues for further research and application of PINNs in the field of computational fluid dynamics.

8 Conclusions

In this work, we introduce a comprehensive training pipeline for physics-informed neural networks, addressing various training pathologies such as spectral bias, imbalanced losses, and causality violation. Our pipeline seamlessly integrates essential techniques, including equation non-dimensionalization, Fourier feature embeddings, loss weighting schemes and causal training strategies. Moreover, we explore additional techniques such as a modified MLP architecture, random weight factorization and curriculum training, which can further improve the training stability and model performance. By sharing our empirical findings, we also provide insights into selecting appropriate hyper-parameters associated with network architectures and learning rate schedules in conjunction with the aforementioned algorithms. To demonstrate

the effectiveness of the proposed training pipeline, we perform thorough ablation studies on a collection of benchmarks which PINNs often struggle with, and showcase the state-of-the-art results, which we believe should serve as a strong baseline for future studies. By establishing these benchmarks, we hope that our contribution will serve as a cornerstone for more fair and systematic comparisons in the development and adoption of PINN-based methodologies, ultimately propelling PINN research towards more effective and reliable solutions in computational science and engineering.

Acknowledgments

We would like to acknowledge support from the US Department of Energy under the Advanced Scientific Computing Research program (grant DE-SC0019116), the US Air Force (grant AFOSR FA9550-20-1-0060), and US Department of Energy/Advanced Research Projects Agency (grant DE-AR0001201). We also thank the developers of the software that enabled our research, including JAX [63], JAX-CFD[70], Matplotlib [71], and NumPy [72].

References

- [1] Athanasios Voulodimos, Nikolaos Doulamis, Anastasios Doulamis, Eftychios Protopapadakis, et al. Deep learning for computer vision: A brief review. *Computational intelligence and neuroscience*, 2018, 2018.
- [2] KR1442 Chowdhary and KR Chowdhary. Natural language processing. *Fundamentals of artificial intelligence*, pages 603–649, 2020.
- [3] Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. MIT press, 2018.
- [4] Tung Nguyen, Johannes Brandstetter, Ashish Kapoor, Jayesh K Gupta, and Aditya Grover. Climax: A foundation model for weather and climate. *arXiv preprint arXiv:2301.10343*, 2023.
- [5] Jaideep Pathak, Shashank Subramanian, Peter Harrington, Sanjeev Raja, Ashesh Chattopadhyay, Morteza Mardani, Thorsten Kurth, David Hall, Zongyi Li, Kamyar Azizzadenesheli, et al. Fourcastnet: A global data-driven high-resolution weather model using adaptive fourier neural operators. *arXiv preprint arXiv:2202.11214*, 2022.
- [6] Remi Lam, Alvaro Sanchez-Gonzalez, Matthew Willson, Peter Wirsberger, Meire Fortunato, Alexander Pritzel, Suman Ravuri, Timo Ewalds, Ferran Alet, Zach Eaton-Rosen, et al. Graphcast: Learning skillful medium-range global weather forecasting. *arXiv preprint arXiv:2212.12794*, 2022.
- [7] Han Wang, Linfeng Zhang, Jiequn Han, and E Weinan. Deepmd-kit: A deep learning package for many-body potential energy representation and molecular dynamics. *Computer Physics Communications*, 228:178–184, 2018.
- [8] D. Pfau, J.S. Spencer, A.G. de G. Matthews, and W.M.C. Foulkes. Ab-initio solution of the many-electron schrödinger equation with deep neural networks. *Phys. Rev. Research*, 2:033429, 2020.
- [9] Kiersten M Ruff and Rohit V Pappu. AlphaFold and implications for intrinsically disordered proteins. *Journal of Molecular Biology*, 433(20):167208, 2021.
- [10] Maziar Raissi, Paris Perdikaris, and George E Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.
- [11] Maziar Raissi, Alireza Yazdani, and George Em Karniadakis. Hidden fluid mechanics: Learning velocity and pressure fields from flow visualizations. *Science*, 367(6481):1026–1030, 2020.
- [12] Luning Sun, Han Gao, Shaowu Pan, and Jian-Xun Wang. Surrogate modeling for fluid flows based on physics-constrained deep learning without simulation data. *Computer Methods in Applied Mechanics and Engineering*, 361:112732, 2020.
- [13] Abhilash Mathews, Manure Francisquez, Jerry W Hughes, David R Hatch, Ben Zhu, and Barrett N Rogers. Uncovering turbulent plasma dynamics via deep learning from partial observations. *Physical Review E*, 104(2):025205, 2021.
- [14] Francisco Sahli Costabal, Yibo Yang, Paris Perdikaris, Daniel E Hurtado, and Ellen Kuhl. Physics-informed neural networks for cardiac activation mapping. *Frontiers in Physics*, 8:42, 2020.
- [15] Georgios Kissas, Yibo Yang, Eileen Hwuang, Walter R Witschey, John A Detre, and Paris Perdikaris. Machine learning in cardiovascular flows modeling: Predicting arterial blood pressure from non-invasive 4D flow MRI data using physics-informed neural networks. *Computer Methods in Applied Mechanics and Engineering*, 358:112623, 2020.
- [16] Zhiwei Fang and Justin Zhan. Deep physical informed neural networks for metamaterial design. *IEEE Access*, 8:24506–24513, 2019.

- [17] Yuyao Chen, Lu Lu, George Em Karniadakis, and Luca Dal Negro. Physics-informed neural networks for inverse problems in nano-optics and metamaterials. *Optics express*, 28(8):11618–11633, 2020.
- [18] Enrui Zhang, Ming Dao, George Em Karniadakis, and Subra Suresh. Analyses of internal structures and defects in materials using physics-informed neural networks. *Science advances*, 8(7):eabk0644, 2022.
- [19] Mahmudul Islam, Md Shajedul Hoque Thakur, Satyajit Mojumder, and Mohammad Nasim Hasan. Extraction of material properties through multi-fidelity deep learning from molecular dynamics simulation. *Computational Materials Science*, 188:110187, 2021.
- [20] Alexander Kovacs, Lukas Exl, Alexander Kornell, Johann Fischbacher, Markus Hovorka, Markus Gusenbauer, Leoni Breth, Harald Oezelt, Masao Yano, Noritsugu Sakuma, et al. Conditional physics informed neural networks. *Communications in Nonlinear Science and Numerical Simulation*, 104:106041, 2022.
- [21] Zhiwei Fang. A high-efficient hybrid physics-informed neural networks based on convolutional neural network. *IEEE Transactions on Neural Networks and Learning Systems*, 33(10):5514–5526, 2021.
- [22] Ehsan Haghighat and Ruben Juanes. Sciann: A keras/tensorflow wrapper for scientific computations and physics-informed deep learning using artificial neural networks. *Computer Methods in Applied Mechanics and Engineering*, 373:113552, 2021.
- [23] Jonthan D Smith, Zachary E Ross, Kamyar Azizzadenesheli, and Jack B Muir. Hyposvi: Hypocentre inversion with stein variational inference and physics informed neural networks. *Geophysical Journal International*, 228(1):698–710, 2022.
- [24] Oliver Hennigh, Susheela Narasimhan, Mohammad Amin Nabian, Akshay Subramaniam, Kaustubh Tangsali, Max Rietmann, Jose del Aguila Ferrandis, Wonmin Byeon, Zhiwei Fang, and Sanjay Choudhry. Nvidia simnetTM: an ai-accelerated multi-physics simulation framework. *arXiv preprint arXiv:2012.07938*, 2020.
- [25] Shengze Cai, Zhicheng Wang, Sifan Wang, Paris Perdikaris, and George Em Karniadakis. Physics-informed neural networks for heat transfer problems. *Journal of Heat Transfer*, 143(6), 2021.
- [26] Sifan Wang, Yujun Teng, and Paris Perdikaris. Understanding and mitigating gradient flow pathologies in physics-informed neural networks. *SIAM Journal on Scientific Computing*, 43(5):A3055–A3081, 2021.
- [27] Sifan Wang, Xinling Yu, and Paris Perdikaris. When and why PINNs fail to train: A neural tangent kernel perspective. *Journal of Computational Physics*, 449:110768, 2022.
- [28] Levi McClenny and Ulisses Braga-Neto. Self-adaptive physics-informed neural networks using a soft attention mechanism. *arXiv preprint arXiv:2009.04544*, 2020.
- [29] Suryanarayana Maddu, Dominik Sturm, Christian L Müller, and Ivo F Sbalzarini. Inverse dirichlet weighting enables reliable training of physics informed neural networks. *Machine Learning: Science and Technology*, 3(1):015026, 2022.
- [30] Mohammad Amin Nabian, Rini Jasmine Gladstone, and Hadi Meidani. Efficient training of physics-informed neural networks via importance sampling. *Computer-Aided Civil and Infrastructure Engineering*, 2021.
- [31] Arka Daw, Jie Bu, Sifan Wang, Paris Perdikaris, and Anuj Karpatne. Rethinking the importance of sampling in physics-informed neural networks. *arXiv preprint arXiv:2207.02338*, 2022.
- [32] Chenxi Wu, Min Zhu, Qinyang Tan, Yadhu Kartha, and Lu Lu. A comprehensive study of non-adaptive and residual-based adaptive sampling for physics-informed neural networks. *Computer Methods in Applied Mechanics and Engineering*, 403:115671, 2023.
- [33] Ameya D Jagtap, Kenji Kawaguchi, and George Em Karniadakis. Adaptive activation functions accelerate convergence in deep and physics-informed neural networks. *Journal of Computational Physics*, 404:109136, 2020.
- [34] Ziqi Liu, Wei Cai, and Zhi-Qin John Xu. Multi-scale deep neural network (MscaledNN) for solving Poisson-Boltzmann equation in complex domains. *arXiv preprint arXiv:2007.11207*, 2020.
- [35] Sifan Wang, Hanwen Wang, and Paris Perdikaris. On the eigenvector bias of fourier feature networks: From regression to solving multi-scale PDEs with physics-informed neural networks. *Computer Methods in Applied Mechanics and Engineering*, 384:113938, 2021.
- [36] Vincent Sitzmann, Julien Martel, Alexander Bergman, David Lindell, and Gordon Wetzstein. Implicit neural representations with periodic activation functions. *Advances in Neural Information Processing Systems*, 33:7462–7473, 2020.
- [37] Han Gao, Luning Sun, and Jian-Xun Wang. Phygeonet: Physics-informed geometry-adaptive convolutional neural networks for solving parameterized steady-state pdes on irregular domain. *Journal of Computational Physics*, 428:110079, 2021.

- [38] Rizal Fathony, Anit Kumar Sahu, Devin Willmott, and J Zico Kolter. Multiplicative filter networks. In *International Conference on Learning Representations*, 2021.
- [39] Ben Moseley, Andrew Markham, and Tarje Nissen-Meyer. Finite basis physics-informed neural networks (fbpinns): a scalable domain decomposition approach for solving differential equations. *arXiv preprint arXiv:2107.07871*, 2021.
- [40] Namgyu Kang, Byeonghyeon Lee, Youngjoon Hong, Seok-Bae Yun, and Eunbyung Park. Pixel: Physics-informed cell representations for fast and accurate pde solvers. *arXiv preprint arXiv:2207.12800*, 2022.
- [41] Chuwei Wang, Shanda Li, Di He, and Liwei Wang. Is l^2 physics informed loss always suitable for training physics informed neural network? *Advances in Neural Information Processing Systems*, 35:8278–8290, 2022.
- [42] Pao-Hsiung Chiu, Jian Cheng Wong, Chinchun Ooi, My Ha Dao, and Yew-Soon Ong. Can-pinn: A fast physics-informed neural network based on coupled-automatic-numerical differentiation method. *Computer Methods in Applied Mechanics and Engineering*, 395:114909, 2022.
- [43] Ehsan Kharazmi, Zhongqiang Zhang, and George Em Karniadakis. hp-vpinns: Variational physics-informed neural networks with domain decomposition. *Computer Methods in Applied Mechanics and Engineering*, 374:113547, 2021.
- [44] Ravi G Patel, Indu Manickam, Nathaniel A Trask, Mitchell A Wood, Myoungkyu Lee, Ignacio Tomas, and Eric C Cyr. Thermodynamically consistent physics-informed neural networks for hyperbolic systems. *Journal of Computational Physics*, 449:110754, 2022.
- [45] Jeremy Yu, Lu Lu, Xuhui Meng, and George Em Karniadakis. Gradient-enhanced physics-informed neural networks for forward and inverse pde problems. *Computer Methods in Applied Mechanics and Engineering*, 393:114823, 2022.
- [46] Hwijae Son, Jin Woo Jang, Woo Jin Han, and Hyung Ju Hwang. Sobolev training for physics informed neural networks. *arXiv preprint arXiv:2101.08932*, 2021.
- [47] Colby L Wight and Jia Zhao. Solving Allen-Cahn and Cahn-Hilliard equations using the adaptive physics informed neural networks. *arXiv preprint arXiv:2007.04542*, 2020.
- [48] Aditi S Krishnapriyan, Amir Gholami, Shandian Zhe, Robert M Kirby, and Michael W Mahoney. Characterizing possible failure modes in physics-informed neural networks. *arXiv preprint arXiv:2109.01050*, 2021.
- [49] Shaan Desai, Marios Mattheakis, Hayden Joy, Pavlos Protopapas, and Stephen Roberts. One-shot transfer learning of physics-informed neural networks. *arXiv preprint arXiv:2110.11286*, 2021.
- [50] Somdatta Goswami, Cosmin Anitescu, Souvik Chakraborty, and Timon Rabczuk. Transfer learning enhanced physics informed neural network for phase-field modeling of fracture. *Theoretical and Applied Fracture Mechanics*, 106:102447, 2020.
- [51] Souvik Chakraborty. Transfer learning based multi-fidelity physics informed deep neural network. *Journal of Computational Physics*, 426:109942, 2021.
- [52] Nasim Rahaman, Aristide Baratin, Devansh Arpit, Felix Draxler, Min Lin, Fred Hamprecht, Yoshua Bengio, and Aaron Courville. On the spectral bias of neural networks. In *International Conference on Machine Learning*, pages 5301–5310, 2019.
- [53] Sifan Wang, Shyam Sankaran, and Paris Perdikaris. Respecting causality is all you need for training physics-informed neural networks. *arXiv preprint arXiv:2203.07404*, 2022.
- [54] Andreas Griewank and Andrea Walther. *Evaluating derivatives: principles and techniques of algorithmic differentiation*. SIAM, 2008.
- [55] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, pages 249–256, 2010.
- [56] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [57] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415*, 2016.
- [58] Zhi-Qin John Xu, Yaoyu Zhang, Tao Luo, Yanyang Xiao, and Zheng Ma. Frequency principle: Fourier analysis sheds light on deep neural networks. *arXiv preprint arXiv:1901.06523*, 2019.
- [59] Ronen Basri, Meirav Galun, Amnon Geifman, David Jacobs, Yoni Kasten, and Shira Kritchman. Frequency bias in neural networks for input of non-uniform density. *arXiv preprint arXiv:2003.04560*, 2020.

- [60] Matthew Tancik, Pratul P Srinivasan, Ben Mildenhall, Sara Fridovich-Keil, Nithin Raghavan, Utkarsh Singhal, Ravi Ramamoorthi, Jonathan T Barron, and Ren Ng. Fourier features let networks learn high frequency functions in low dimensional domains. *arXiv preprint arXiv:2006.10739*, 2020.
- [61] Sifan Wang, Hanwen Wang, Jacob H Seidman, and Paris Perdikaris. Random weight factorization improves the training of continuous neural representations. *arXiv preprint arXiv:2210.01274*, 2022.
- [62] Tim Salimans and Durk P Kingma. Weight normalization: A simple reparameterization to accelerate training of deep neural networks. *Advances in neural information processing systems*, 29, 2016.
- [63] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Neca, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018.
- [64] Suchuan Dong and Naxian Ni. A method for representing periodic functions and enforcing exactly periodic boundary conditions with deep neural networks. *Journal of Computational Physics*, 435:110242, 2021.
- [65] N Sukumar and Ankit Srivastava. Exact imposition of boundary conditions with distance functions in physics-informed deep neural networks. *arXiv preprint arXiv:2104.08426*, 2021.
- [66] Lu Lu, Raphael Pestourie, Wenjie Yao, Zhicheng Wang, Francesc Verdugo, and Steven G Johnson. Physics-informed neural networks with hard constraints for inverse design. *arXiv preprint arXiv:2102.04626*, 2021.
- [67] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [68] U Ghia, K.N Ghia, and C.T Shin. High-re solutions for incompressible flow using the navier-stokes equations and a multigrid method. *Journal of Computational Physics*, 48(3):387–411, 1982.
- [69] Pi-Yueh Chuang and Lorena A Barba. Experience report of physics-informed neural networks in fluid simulations: pitfalls and frustration. *arXiv preprint arXiv:2205.14249*, 2022.
- [70] Dmitrii Kochkov, Jamie A. Smith, Ayya Alieva, Qing Wang, Michael P. Brenner, and Stephan Hoyer. Machine learning–accelerated computational fluid dynamics. *Proceedings of the National Academy of Sciences*, 118(21), 2021.
- [71] John D Hunter. Matplotlib: A 2D graphics environment. *IEEE Annals of the History of Computing*, 9(03):90–95, 2007.
- [72] Charles R Harris, K Jarrod Millman, Stéfan J van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J Smith, et al. Array programming with numpy. *Nature*, 585(7825):357–362, 2020.
- [73] Yuan Cao, Zhiying Fang, Yue Wu, Ding-Xuan Zhou, and Quanquan Gu. Towards understanding the spectral bias of deep learning. *arXiv preprint arXiv:1912.01198*, 2019.
- [74] Basri Ronen, David Jacobs, Yoni Kasten, and Shira Kritchman. The convergence rate of neural networks for learned functions of different frequencies. In *Advances in Neural Information Processing Systems*, pages 4761–4771, 2019.
- [75] Arthur Jacot, Franck Gabriel, and Clément Hongler. Neural tangent kernel: Convergence and generalization in neural networks. In *Advances in neural information processing systems*, pages 8571–8580, 2018.
- [76] Sanjeev Arora, Simon S Du, Wei Hu, Zhiyuan Li, Russ R Salakhutdinov, and Ruosong Wang. On exact computation with an infinitely wide neural net. In *Advances in Neural Information Processing Systems*, pages 8141–8150, 2019.
- [77] Jaehoon Lee, Lechao Xiao, Samuel Schoenholz, Yasaman Bahri, Roman Novak, Jascha Sohl-Dickstein, and Jeffrey Pennington. Wide neural networks of any depth evolve as linear models under gradient descent. In *Advances in neural information processing systems*, pages 8572–8583, 2019.
- [78] Arieh Iserles. *A first course in the numerical analysis of differential equations*. Number 44. Cambridge university press, 2009.

A Spectral Bias through the lens of the Neural Tangent Kernel

We investigate spectral bias [52, 73, 74] in the training behavior of deep fully-connected networks through the lens of Neural Tangent Kernel(NTK) theory. Let $f_\theta(\mathbf{x})$ be a scalar-valued fully-connected neural network. Given a training

data-set $\{\mathbf{X}_{\text{train}}, \mathbf{Y}_{\text{train}}\}$, where $\mathbf{X}_{\text{train}} = (\mathbf{x}_i)_{i=1}^N$ are inputs and $\mathbf{Y}_{\text{train}} = (y_i)_{i=1}^N$ are the corresponding labels. We consider a network trained by minimizing the mean square loss

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N |f_{\theta}(\mathbf{x}_i) - y_i|^2. \quad (\text{A.1})$$

Following the derivation of Jacot *et al.* [75, 76], we can define the resulting neural tangent kernel $\mathbf{K}$, whose ij -th entry is given by

$$\mathbf{K}_{ij} = \mathbf{K}(\mathbf{x}_i, \mathbf{x}_j) = \left\langle \frac{\partial f_{\theta}(\mathbf{x}_i)}{\partial \theta}, \frac{\partial f_{\theta}(\mathbf{x}_j)}{\partial \theta} \right\rangle. \quad (\text{A.2})$$

The NTK theory shows that, under gradient descent dynamics with an infinitesimally small learning rate (gradient flow), the kernel $\mathbf{K}$ converges to a deterministic kernel $\mathbf{K}^*$ and does not change during training as the width of the network grows to infinity.

Furthermore, under the asymptotic conditions stated in Lee *et al.* [77], we may derive that

$$\frac{df_{\theta(\tau)}(\mathbf{X}_{\text{train}})}{d\tau} \approx -\mathbf{K} \cdot (f_{\theta(\tau)}(\mathbf{X}_{\text{train}}) - \mathbf{Y}_{\text{train}}), \quad (\text{A.3})$$

where $\theta(\tau)$ denotes the parameters of the network at iteration τ and $f_{\theta(\tau)}(\mathbf{X}_{\text{train}}) = (f_{\theta(\tau)}(\mathbf{x}_i))_{i=1}^N$. Then, it directly follows that

$$f_{\theta(\tau)}(\mathbf{X}_{\text{train}}) \approx (I - e^{-\mathbf{K}\tau}) \cdot \mathbf{Y}_{\text{train}}. \quad (\text{A.4})$$

Since the kernel $\mathbf{K}$ is positive semi-definite, we can take its spectral decomposition $\mathbf{K} = \mathbf{Q}^T \mathbf{\Lambda} \mathbf{Q}$, where $\mathbf{Q}$ is an orthogonal matrix whose i -th column is the eigenvector $\mathbf{q}_i$ of $\mathbf{K}$ and $\mathbf{\Lambda}$ is a diagonal matrix whose diagonal entries λ_i are the corresponding eigenvalues. Since $e^{-\mathbf{K}\tau} = \mathbf{Q}e^{-\mathbf{\Lambda}\tau}\mathbf{Q}^T$, we have

$$\mathbf{Q}^T (f_{\theta(\tau)}(\mathbf{X}_{\text{train}}) - \mathbf{Y}_{\text{train}}) = -e^{\mathbf{\Lambda}\tau} \mathbf{Q}^T \mathbf{Y}_{\text{train}}, \quad (\text{A.5})$$

which implies

$$\begin{bmatrix} \mathbf{q}_1^T \\ \mathbf{q}_2^T \\ \vdots \\ \mathbf{q}_N^T \end{bmatrix} (f_{\theta(\tau)}(\mathbf{X}_{\text{train}}) - \mathbf{Y}_{\text{train}}) = \begin{bmatrix} e^{-\lambda_1 \tau} & & & \\ & e^{-\lambda_2 \tau} & & \\ & & \ddots & \\ & & & e^{-\lambda_N \tau} \end{bmatrix} \begin{bmatrix} \mathbf{q}_1^T \\ \mathbf{q}_2^T \\ \vdots \\ \mathbf{q}_N^T \end{bmatrix} \mathbf{Y}_{\text{train}}. \quad (\text{A.6})$$

The above equation shows that the convergence rate of $\mathbf{q}_i^T (f_{\theta(\tau)}(\mathbf{X}_{\text{train}}) - \mathbf{Y}_{\text{train}})$ is determined by the i -th eigenvalue λ_i . Moreover, we can decompose the training error into the eigen-space of the NTK as

$$f_{\theta(\tau)}(\mathbf{X}_{\text{train}}) - \mathbf{Y}_{\text{train}} = \sum_{i=1}^N (f_{\theta(\tau)}(\mathbf{X}_{\text{train}}) - \mathbf{Y}_{\text{train}}, \mathbf{q}_i) \mathbf{q}_i \quad (\text{A.7})$$

$$= \sum_{i=1}^N \mathbf{q}_i^T (f_{\theta(\tau)}(\mathbf{X}_{\text{train}}) - \mathbf{Y}_{\text{train}}) \mathbf{q}_i \quad (\text{A.8})$$

$$= \sum_{i=1}^N (e^{-\lambda_i \tau} \mathbf{q}_i^T \mathbf{Y}_{\text{train}}) \mathbf{q}_i. \quad (\text{A.9})$$

Clearly, the network is biased to first learn the target function along the eigen-directions of neural tangent kernel with larger eigenvalues, and then the remaining components corresponding to smaller eigenvalues. Cao *et al.* [73] provide a detailed analysis of the convergence rate of these components. For conventional fully-connected neural networks, the eigenvalues of the NTK shrink monotonically as the frequency of the corresponding eigenfunctions increases, yielding a significantly lower convergence rate for high frequency components of the target function [52, 74]. This indeed reveals the so-called ‘‘spectral bias’’ [52] pathology of deep neural networks. More importantly, we may conclude that the eigen-space of the neural tangent kernel characterizes the learnability of a target function by a neural network.

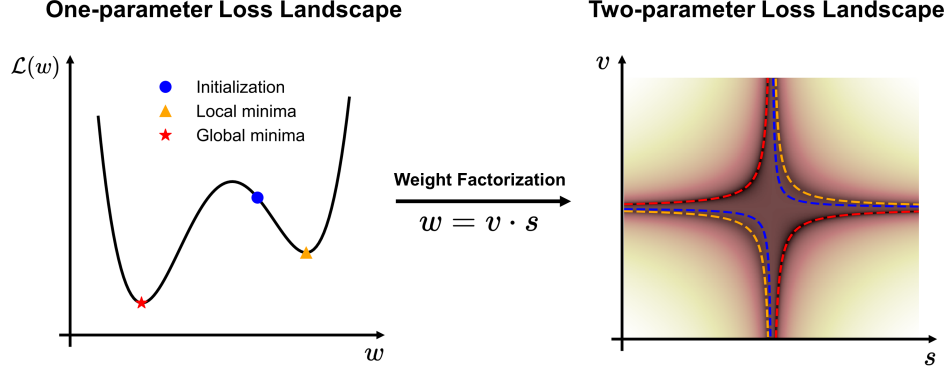


Figure 18: Weight factorization transforms loss landscapes and shortens the distance to minima.

B Random Weight Factorization

In this section, we provide an intuitive understanding and some theoretical explanations of random weight factorization. For additional numerical validations of RWF, please see [61].

To provide an intuitive understanding, let us consider the simplest setting of a one-parameter loss function $\ell(w)$. In this case, the weight factorization can be simplified to $w = s \cdot v$ with two scalars s and v . It is important to note that for any given $w \neq 0$, there exist infinitely many pairs (s, v) such that $w = s \cdot v$. These pairs form a family of hyperbolas in the sv -plane, with one for each choice of signs for both s and v . Consequently, the loss function in the sv -plane remains constant along these hyperbolas.

Figure 18 gives a visual illustration of the difference between the original loss landscape as a function of w versus the loss landscape in the factorized sv -plane. In the left panel, we plot the original loss function as well as an initial parameter point, the local minimum, and the global minimum. The right panel shows how in the factorized parameter space, each of these three points corresponds to two hyperbolas in the sv -plane. Note how the distance between the initialization and the global minima is reduced from the left to the right panel upon an appropriate choice of factorization. The key observation is that the distance between factorizations representing the initial parameter and the global minimum becomes arbitrarily small in the sv -plane for larger values of s . Indeed, we can prove that this holds for any general loss function in arbitrary parameter dimensions. Further details can be found in [35].

Theorem B.1. Suppose that $\mathcal{L}(\theta)$ is the associated loss function of a neural network defined in (4.1) and (4.2). For a given θ , we define U_θ as the set containing all possible weight factorizations

$$U_\theta = \left\{ (\mathbf{s}^{(l)}, \mathbf{V}^{(l)})_{l=1}^{L+1} : \text{diag}(\mathbf{s}^{(l)}) \cdot \mathbf{V}^{(l)} = \mathbf{W}^{(l)}, \quad l = 1, \dots, L+1 \right\}. \quad (\text{B.1})$$

Then for any θ, θ' , we have

$$\text{dist}(U_\theta, U_{\theta'}) := \min_{\mathbf{x} \in U_\theta, \mathbf{y} \in U_{\theta'}} \|\mathbf{x} - \mathbf{y}\| = 0. \quad (\text{B.2})$$

Proof. Starting from any fixed network parameters $\theta = \{\mathbf{W}^{(l)}, \mathbf{b}^l\}_{l=1}^{L+1}$, we consider the following weight factorization

$$\text{diag}(\mathbf{s}^{(l)}) \cdot \mathbf{V}^{(l)} = \mathbf{W}^{(l)}, \quad l = 1, 2, \dots, L+1. \quad (\text{B.3})$$

Next, consider the set of all possible weight factorizations associated with the initialization θ as

$$U_\theta = \left\{ (\mathbf{s}^{(l)}, \mathbf{V}^{(l)})_{l=1}^{L+1} : \text{diag}(\mathbf{s}^{(l)}) \cdot \mathbf{V}^{(l)} = \mathbf{W}^{(l)}, \quad l = 1, \dots, L+1 \right\}. \quad (\text{B.4})$$

Let us now define U_0 in the factorized parameter space by

$$U_0 = \{(\mathbf{s}^{(l)}, \mathbf{0})_{l=1}^{L+1} : \mathbf{s}^{(l)} \in \mathbb{R}^{d_l}, \quad l = 1, \dots, L+1\}. \quad (\text{B.5})$$

Since the network parameters θ are fixed, there exists a constant $C(\theta)$ such that

$$\|\mathbf{V}^{(l)}\| \leq \frac{\|\mathbf{W}^{(l)}\|}{\|\mathbf{s}^{(l)}\|} \leq \frac{C(\theta)}{\|\mathbf{s}^{(l)}\|}, \quad l = 1, \dots, L+1. \quad (\text{B.6})$$

For any weight factorization $(\mathbf{s}^{(l)}, \mathbf{V}^{(l)})_{l=1}^{L+1}$, we can take $(\mathbf{s}^{(l)}, 0)_{l=1}^{L+1} \in U_*$. By the definition of distance between sets, we obtain

$$\text{dist}(U_\theta, U_0) = \min_{\mathbf{x} \in U_\theta, \mathbf{y} \in U_*} \|\mathbf{x} - \mathbf{y}\| \leq \sqrt{\sum_{l=1}^{L+1} \|\mathbf{V}^{(l)}\|^2} \leq C(\theta) \sqrt{\sum_{l=1}^{L+1} \frac{1}{\|\mathbf{s}^{(l)}\|^2}}. \quad (\text{B.7})$$

Therefore, for any network parameters θ, θ' , taking $C = \max\{C(\theta), C(\theta')\}$ yields

$$\text{dist}(U_\theta, U_{\theta'}) \leq \text{dist}(U_\theta, U_0) + \text{dist}(U_0, U_{\theta'}) \leq 2C \sqrt{\sum_{l=1}^{L+1} \frac{1}{\|\mathbf{s}^{(l)}\|^2}}. \quad (\text{B.8})$$

For $l = 1, \dots, L+1$, letting $\mathbf{s}^{(l)}$ go to infinity, we have

$$\text{dist}(U_\theta, U_{\theta'}) = 0 \quad (\text{B.9})$$

As a corollary, let θ denote a network initialization and θ_* be a proper local minimum, then there exists a weight factorization with large enough scale factors $\mathbf{s}$, such that the distance between θ and θ_* can be arbitrarily small in the factorized parameter space. $\square$

A different way to examine the effect of the proposed weight factorization is by studying its associated gradient updates. Recall that a standard gradient descent update with a learning rate η takes the form

$$\mathbf{w}_{n+1}^{(k,l)} = \mathbf{w}_n^{(k,l)} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{w}_n^{(k,l)}}. \quad (\text{B.10})$$

The following theorem derives the corresponding gradient descent update expressed in the original parameter space for models using the proposed weight factorization.

Theorem B.2. *Under the weight factorization of (4.4), the gradient descent update is given by*

$$\mathbf{w}_{n+1}^{(k,l)} = \mathbf{w}_n^{(k,l)} - \eta \left([s_n^{(k,l)}]^2 + \|\mathbf{v}_n^{(k,l)}\|_2^2 \right) \frac{\partial \mathcal{L}}{\partial \mathbf{w}_n^{(k,l)}} + \mathcal{O}(\eta^2), \quad (\text{B.11})$$

for $l = 1, 2, \dots, L+1$ and $k = 1, 2, \dots, d_l$.

By comparing (B.10) and (B.11), we observe that the weight factorization $\mathbf{w} = \mathbf{s} \cdot \mathbf{v}$ re-scales the learning rate of $\mathbf{w}$ by a factor of $(s^2 + \|\mathbf{v}\|_2^2)$. Since $\mathbf{s}, \mathbf{v}$ are trainable parameters, this analysis suggests that this weight factorization effectively assigns a self-adaptive learning rate to each neuron in the network.

Proof. Suppose that $f^{(k,l)}$ denotes k -th component of $\mathbf{f}^{(l)} \in \mathbb{R}^{d_l}$. Under the proposed weight factorization in (4.4), differentiating the loss function $\mathcal{L}$ with respect to $\mathbf{w}^{(k,l)}$ and $s^{(k,l)}$, respectively, yields

$$s_{n+1}^{(k,l)} = s_n^{(k,l)} - \eta \frac{\partial \mathcal{L}}{\partial s_n^{(k,l)}} = s_n^{(k,l)} - \eta \frac{\partial \mathcal{L}}{\partial f^{(k,l)}} \cdot \mathbf{v}_n^{(k,l)} \cdot \mathbf{g}^{(l-1)}, \quad (\text{B.12})$$

$$\mathbf{v}_{n+1}^{(k,l)} = \mathbf{v}_n^{(k,l)} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{v}_n^{(k,l)}} = \mathbf{v}_n^{(k,l)} - \eta s_n^{(k,l)} \frac{\partial \mathcal{L}}{\partial f^{(k,l)}} \cdot \mathbf{g}^{(l-1)}. \quad (\text{B.13})$$

Note that

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}_n^{(k,l)}} = \frac{\partial \mathcal{L}}{\partial f^{(k,l)}} \cdot \mathbf{g}^{(l-1)}, \quad (\text{B.14})$$

and the update rule of $\mathbf{v}^{(k,l)}$ and $s^{(k,l)}$ can be re-written as

$$s_{n+1}^{(k,l)} = s_n^{(k,l)} - \eta \mathbf{v}_n^{(k,l)} \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{w}_n^{(k,l)}}, \quad (\text{B.15})$$

$$\mathbf{v}_{n+1}^{(k,l)} = \mathbf{v}_n^{(k,l)} - \eta s_n^{(k,l)} \frac{\partial \mathcal{L}}{\partial \mathbf{w}_n^{(k,l)}}. \quad (\text{B.16})$$

Since $\mathbf{w}^{(k,l)} = s^{(k,l)} \cdot \mathbf{v}^{(k,l)}$, the update rule of $\mathbf{w}^{(k,l)}$ is given by

$$\mathbf{w}_{n+1}^{(k,l)} = \mathbf{w}_n^{(k,l)} - \eta \left([s_n^{(k,l)}]^2 + \|\mathbf{v}_n^{(k,l)}\|_2^2 \right) \frac{\partial \mathcal{L}}{\partial \mathbf{w}_n^{(k,l)}} + \mathcal{O}(\eta^2). \quad (\text{B.17})$$

$\square$

C PINNs can violate causality

To illustrate that PINNs can violate causality, we closely examine the minimization of the PDE residual loss $\mathcal{L}_r$ (see equation 2.8). Before doing so, let us introduce some notation for convenience. Suppose that $0 = t_1 < t_2 < \dots < t_{N_t} = T$ discretizes the temporal domain, and $\{\mathbf{x}_j\}_{j=1}^{N_x}$ discretizes the spatial domain Ω . Now, for a given collection of spatial locations $\{\mathbf{x}_j\}_{j=1}^{N_x}$, we can define the temporal residual loss as

$$\mathcal{L}_r(t, \theta) = \frac{1}{N_x} \sum_{j=1}^{N_x} \left| \frac{\partial \mathbf{u}_\theta}{\partial t}(t, \mathbf{x}_j) + \mathcal{N}[\mathbf{u}_\theta](t, \mathbf{x}_j) \right|^2, \quad (\text{C.1})$$

For a specified set of parameters θ and a short time interval $[t^*, t^* + \Delta t]$, the PDE residual loss $\mathcal{L}_r(t^*, \theta)$ essentially measures the deviation from the solution of the corresponding PDE

$$\mathcal{N}[\mathbf{u}](t, x) = 0, \quad (t, x) \in [t^*, t^* + \Delta t] \times \Omega \quad (\text{C.2})$$

$$u(t^*, x) = u_\theta(t^*, x), \quad x \in \Omega \quad (\text{C.3})$$

Here, $u_\theta(t^*, x)$ represents the network's prediction using the given fixed parameters θ evaluated at t^* . As a result, even if $\mathcal{L}_r(t^*, \theta) = 0$, the accuracy of the predicted solution in $[t^*, t^* + \Delta t]$ is determined by the deviation of $u_\theta(t^*, x)$ from the ground truth and thus the error will propagate along the time. Hence, we argue that the temporal residual loss $\mathcal{L}_r(t, \theta)$ should be based on the current predicted solution at time t and the optimization process is meaningful only if the predicted solution is reasonable for previous times.

Note that the residual loss 2.8 can be rewritten as

$$\mathcal{L}_r(\theta) = \frac{1}{N_t} \sum_{i=1}^{N_t} \mathcal{L}_r(t_i, \theta) \quad (\text{C.4})$$

$$= \frac{1}{N_t N_x} \sum_{i=1}^{N_t} \sum_{j=1}^{N_x} \left| \frac{\partial \mathbf{u}_\theta}{\partial t}(t_i, \mathbf{x}_j) + \mathcal{N}[\mathbf{u}_\theta](t_i, \mathbf{x}_j) \right|^2. \quad (\text{C.5})$$

Next, we discretize $\frac{\partial \mathbf{u}_\theta}{\partial t}$ using the forward Euler scheme [78]. For any $1 \leq i \leq N_t - 1$, $\mathcal{L}(t_i, \theta)$ can be approximated by

$$\begin{aligned} \mathcal{L}_r(t_i, \theta) &\approx \frac{1}{N_x} \sum_{j=1}^{N_x} \left| \frac{\mathbf{u}_\theta(t_i, \mathbf{x}_j) - \mathbf{u}_\theta(t_{i-1}, \mathbf{x}_j)}{\Delta t} + \mathcal{N}[\mathbf{u}_\theta](t_i, \mathbf{x}_j) \right|^2 \\ &\approx \frac{|\Omega|}{\Delta t^2} \int_{\Omega} |\mathbf{u}_\theta(t_i, \mathbf{x}) - \mathbf{u}_\theta(t_{i-1}, \mathbf{x}) + \Delta t \mathcal{N}[\mathbf{u}_\theta](t_i, \mathbf{x})|^2 d\mathbf{x}. \end{aligned} \quad (\text{C.6})$$

From the above expression, we immediately obtain that the proper minimization of $\mathcal{L}_r(t_i, \theta)$ should be based on the correct prediction of both $\mathbf{u}_\theta(t_i, \mathbf{x})$ and $\mathbf{u}_\theta(t_{i-1}, \mathbf{x})$, while the original formulation tends to minimize all $\mathcal{L}_r(t_i, \theta)$ simultaneously. As a result, the residual loss $\mathcal{L}_r(t_i, \theta)$ will be minimized even if the predictions at t_i and previous times are inaccurate. This behavior inevitably violates temporal causality, making the PINN model susceptible to learn erroneous solutions.

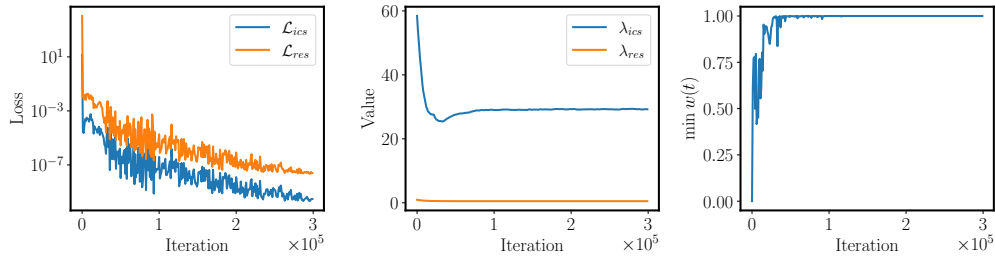


Figure 19: Allen-Cahn equation: *left*: Loss convergence of the initial condition loss $\mathcal{L}_{ics}$ and the PDE residual loss $\mathcal{L}_{res}$ during training. *Mid*: Changes of loss weights during training. *Right*: Minimum value of the temporal residual $\min_t w(t)$ during training. All temporal PDE residuals are properly minimized if $\min_t w(t)$ converge to 1.

Table 6: *Allen-Cahn equation*: Hyper-parameter configuration.

Parameter	Value
Architecture Parameters	
Architecture	Modified MLP
Number of layers	4
Layer size	256
Activation	Tanh
Fourier feature scale	2.0
RWF	$\mu = 0.5, \sigma = 0.1$
Training Parameters	
Learning rate	0.001
Decay steps	5,000
Training steps	300,000
Batch size	8,192
Weighting Parameters	
Weighting scheme	NTK
Causal tolerance	1.0
Number of chunks	32

Table 7: *Advection equation*: Hyper-parameter configuration.

Parameter	Value
Architecture Parameters	
Architecture	Modified MLP
Number of layers	4
Layer size	256
Activation	Tanh
Fourier feature scale	1.0
RWF	$\mu = 1.0, \sigma = 0.1$
Training Parameters	
Learning rate	0.001
Decay steps	2,000
Training steps	200,000
Batch size	8,192
Weighting Parameters	
Weighting scheme	Grad Norm
Causal tolerance	1.0
Number of chunks	32

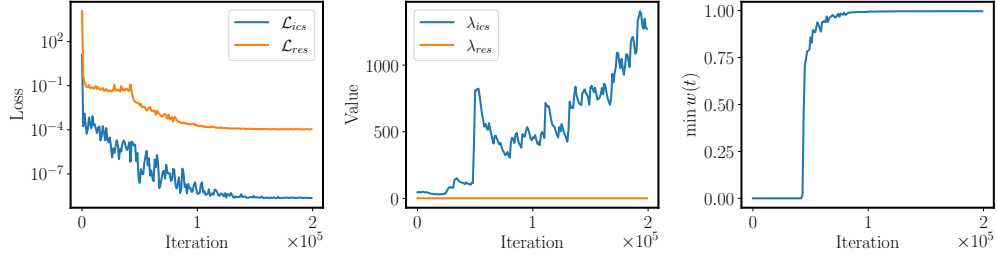


Figure 20: *Advection equation*: left: Loss convergence of the initial condition loss $\mathcal{L}_{ics}$ and the PDE residual loss $\mathcal{L}_{res}$ during training. Mid: Changes of loss weights during training. Right: Minimum value of the temporal residual $\min_t w(t)$ during training. All temporal PDE residuals are properly minimized if $\min_t w(t)$ converge to 1.

Table 8: *Stokes equation*: Hyper-parameter configuration.

Parameter	Value
Architecture Parameters	
Architecture	Modified MLP
Number of layers	4
Layer size	256
Activation	GeLU
Fourier feature scale	10.0
RWF	$\mu = 0.5, \sigma = 0.1$
Training Parameters	
Learning rate	0.001
Decay steps	2,000
Training steps	100,000
Batch size	8,192
Weighting Parameters	
Weighting scheme	Grad Norm

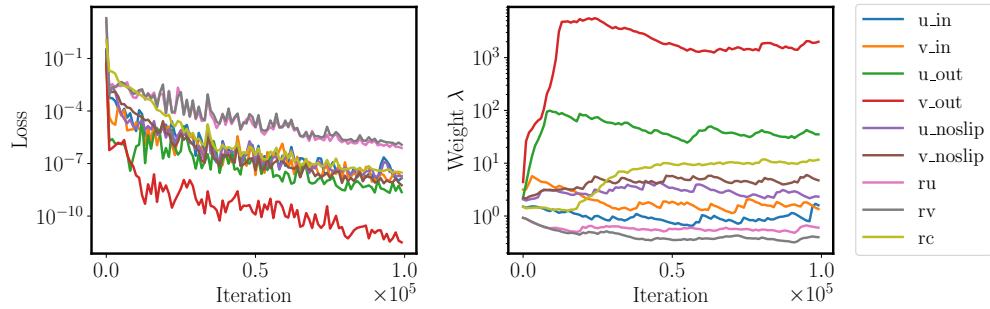


Figure 21: *Stokes equation*: left: Loss convergence of the initial condition loss $\mathcal{L}_{ics}$ and the PDE residual loss $\mathcal{L}_{res}$ during training. Right: Changes of loss weights during training.

Table 9: *Kuramoto–Sivashinsky equation*: Hyper-parameter configuration.

Parameter	Value
Architecture Parameters	
Architecture	Modified MLP
Number of layers	5
Layer size	256
Activation	Tanh
Fourier feature scale	1.0
RWF	$\mu = 0.5, \sigma = 0.1$
Training Parameters	
Learning rate	0.001
Decay steps	2,000
Number of time windows	10
Training steps per window	200,000
Batch size	4,096
Weighting Parameters	
Weighting scheme	Grad Norm
Causal tolerance	1.0
Number of chunks	16

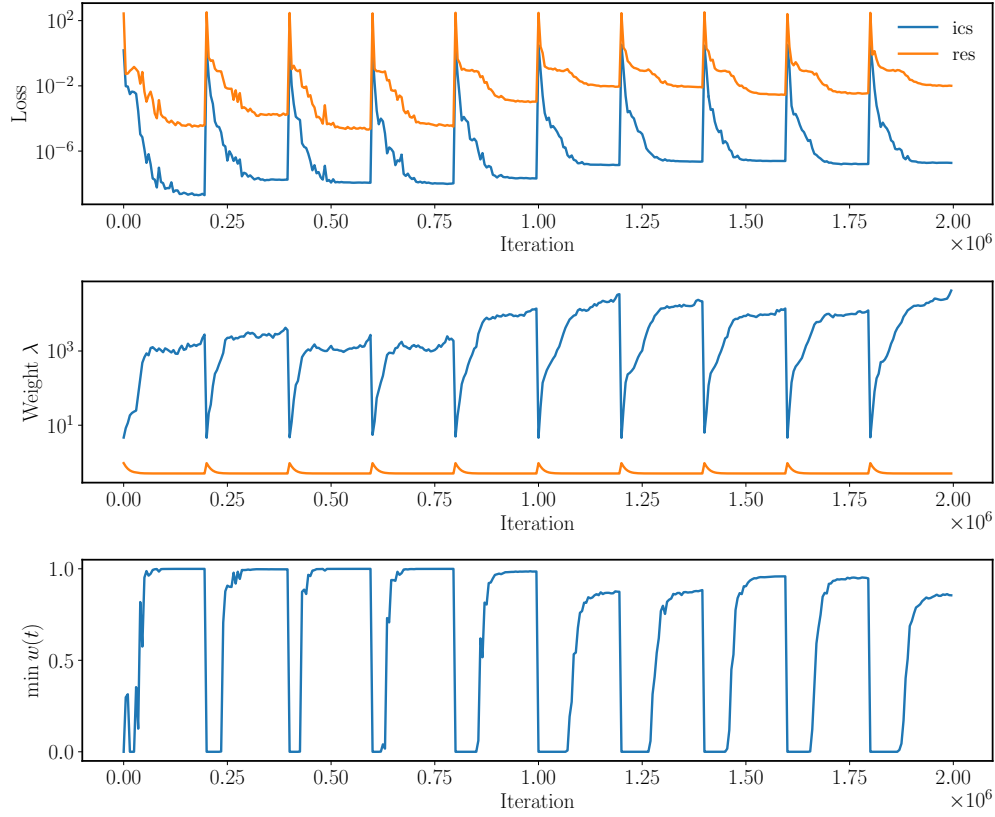


Figure 22: *Kuramoto–Sivashinsky equation*: *Top*: Loss convergence of the initial condition loss $\mathcal{L}_{ics}$ and the PDE residual loss $\mathcal{L}_{res}$ during training. *Mid*: Changes of loss weights during training. *Bottom*: Changes of causal weights during training.

Table 10: *Lid-driven cavity flow*: Hyper-parameter configuration.

Parameter	Value
Architecture Parameters	
Architecture	Modified MLP
Number of layers	5
Layer size	256
Activation	Tanh
Fourier feature scale	10.0
RWF	$\mu = 1.0, \sigma = 0.1$
Training Parameters	
Learning rate	0.001
Decay steps	10,000
Batch size	8,192
Curriculum Training	
Re	[100, 400, 1,000, 3,200]
Training steps	[50,000, 50,000, 10,0000, 50,0000]
Weighting Parameters	
Weighting scheme	Grad Norm

Ablation Settings				Performance	
Fourier Feature	RWF	Grad Norm	Modified MLP	Rel. L^2 error	Run time (min)
✓	✓	✓	✓	1.34×10^{-1}	58.86
✗	✓	✓	✓	7.32×10^{-1}	51.28
✓	✗	✓	✓	1.59×10^{-1}	62.01
✓	✓	✗	✓	3.38×10^{-1}	57.16
✓	✓	✓	✗	5.48×10^{-1}	23.40
✗	✗	✗	✗	7.94×10^{-1}	17.96

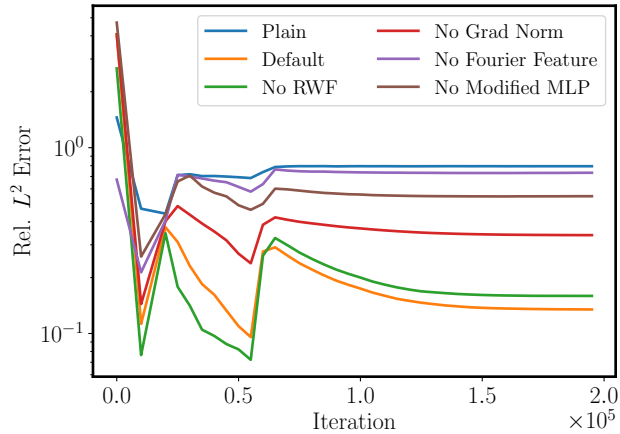
Table 11: *Lid-driven cavity flow*: Relative L^2 error and run time for an ablation study illustrating the impact of disabling nondimensionalization and individual components of the proposed training pipeline. The error is measured against the norm of flow velocity $\|\mathbf{u}\|_2 = \sqrt{u^2 + v^2}$. The first row corresponds to the fine-tuned model using Modified MLP whose hyper-parameter configuration can be found in Table 10.Figure 23: *Lid-driven cavity*: Convergence of relative L^2 error for the ablation study with different components disabled.

Table 12: *Navier-Stokes flow in torus*: Hyper-parameter configuration.

Parameter	Value
Architecture Parameters	
Architecture	Modified MLP
Number of layers	4
Layer size	256
Activation	Tanh
Fourier feature scale	1.0
RWF	$\mu = 0.5, \sigma = 0.1$
Training Parameters	
Learning rate	0.001
Decay steps	2,000
Number of time windows	5
Training steps per window	150,000
Batch size	8,192
Weighting Parameters	
Weighting scheme	Grad Norm
Causal tolerance	1.0
Number of chunks	16

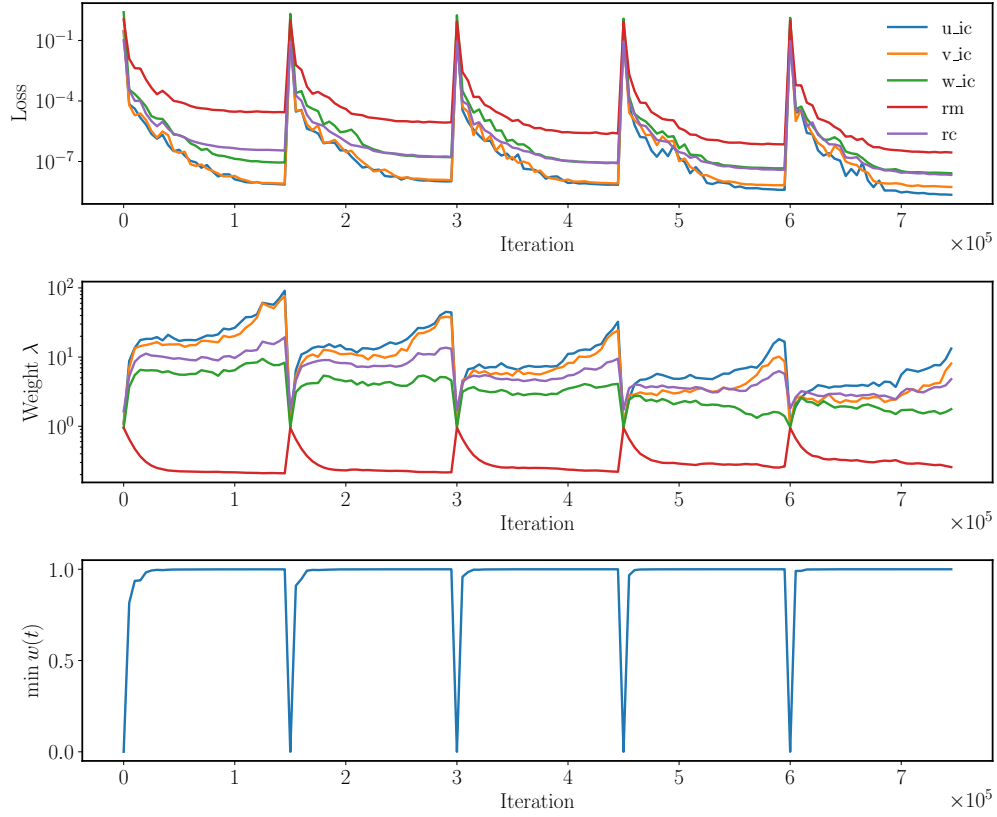


Figure 24: *Navier-Stokes flow in torus*: *Top*: Loss convergence of the initial condition loss $\mathcal{L}_{ics}$ and the PDE residual loss $\mathcal{L}_{res}$ during training. *Mid*: Changes of loss weights during training. *Bottom*: Changes of causal weights during training.

Table 13: *Navier-Stokes flow around a cylinder: Hyper-parameter configuration.*

Parameter	Value
Architecture Parameters	
Architecture	Modified MLP
Number of layers	5
Layer size	256
Activation	Tanh
Fourier feature scale	1.0
RWF	$\mu = 1.0, \sigma = 0.1$
Training Parameters	
Learning rate	0.001
Decay steps	2,000
Number of time windows	10
Training steps per window	200,000
Batch size	8,192
Weighting Parameters	
Weighting scheme	Grad Norm
Causal tolerance	1.0
Number of chunks	16

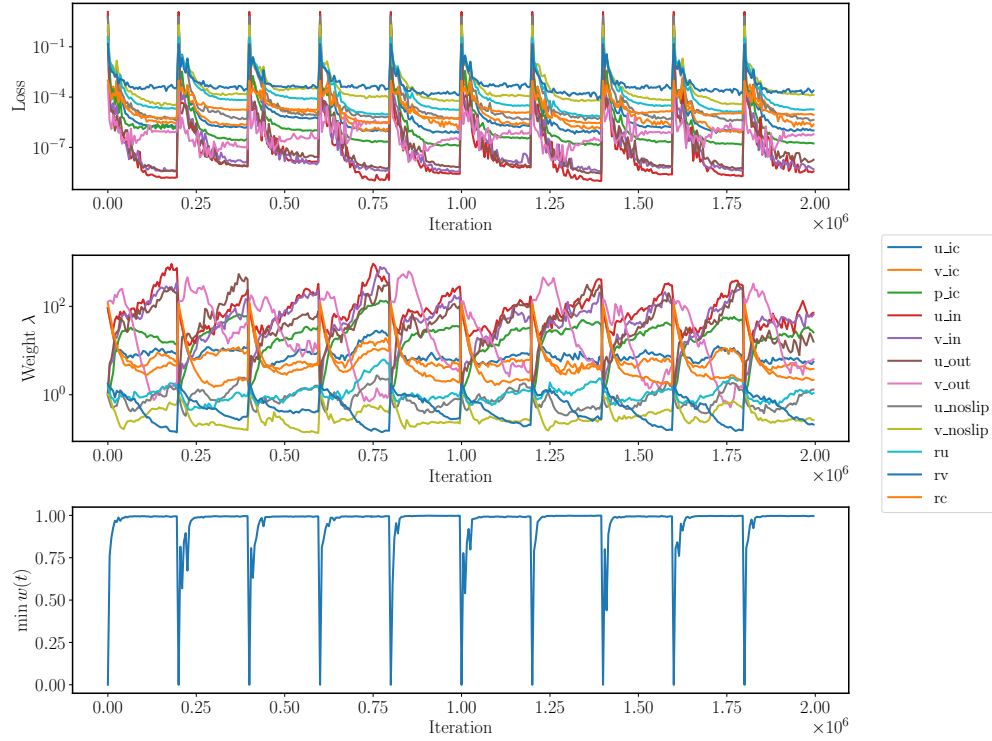


Figure 25: *Navier-Stokes flow around cylinder: Top: Loss convergence of the initial condition loss $\mathcal{L}_{ics}$ and the PDE residual loss $\mathcal{L}_{res}$ during training. Mid: Changes of loss weights during training. Bottom: Changes of causal weights during training.*